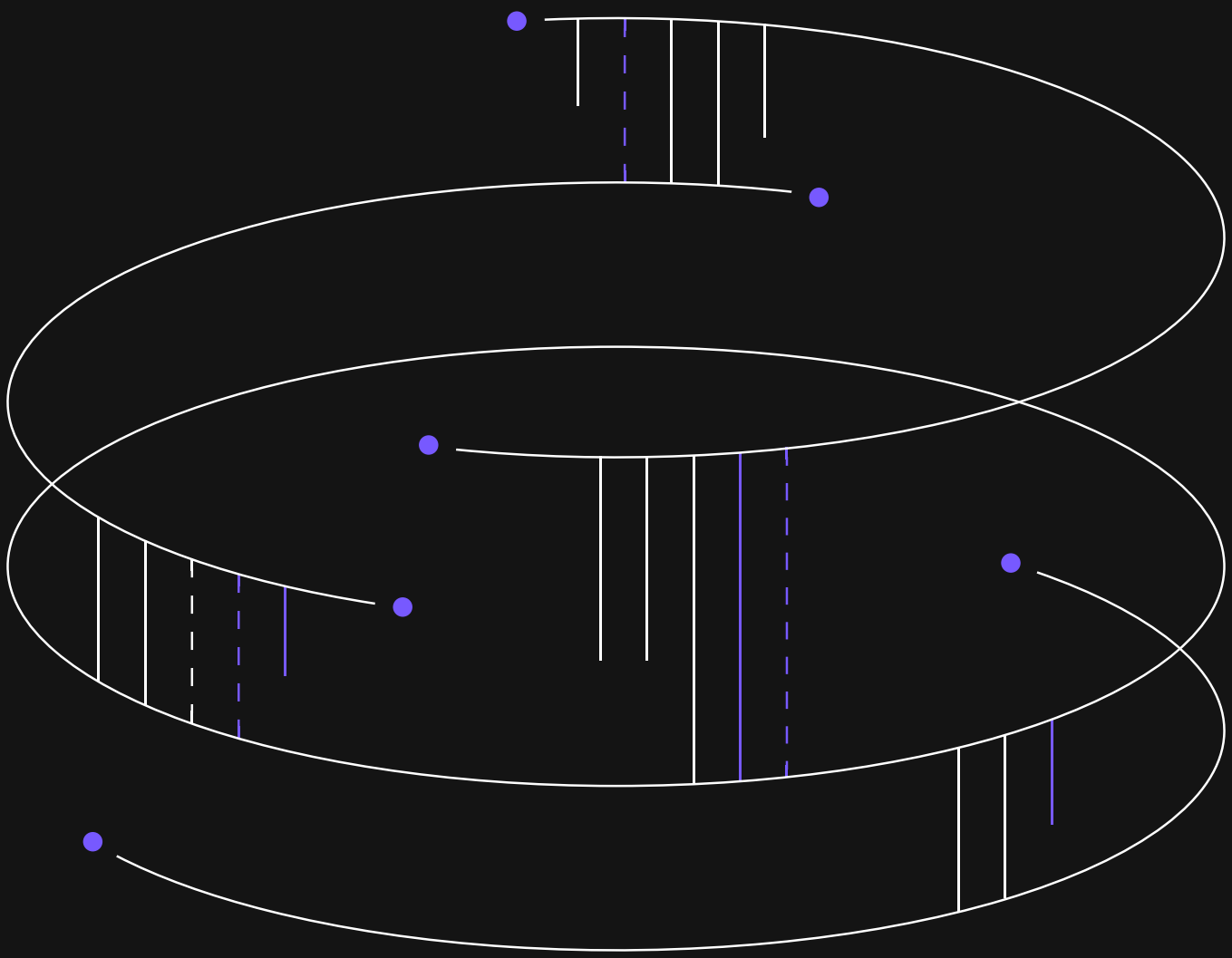
Машинное обучение

Лонгрид

Неделя 6

Деревья решений



Содержание

1. **Что такое деревья решений** ... 4
   - 1.1. Графическое представление ... 6
2. **Виды деревьев решений** ... 7
3. **Технические аспекты дерева решений** ... 9
   - 3.1. Определение дерева решений ... 9
   - 3.2. Функции в узлах ... 10
   - 3.3. Метки в листьях ... 12
   - 3.4. Разбиение пространства признаков ... 12
   - 3.5. Предсказательная функция дерева ... 13
   - 3.6. Выводы ... 13
4. **Обучение дерева решений** ... 14
   - 4.1. Нормировка неопределённостей ... 18
   - 4.2. Структура построения дерева ... 19
5. **Обучение дерева регрессии** ... 21
6. **Обучение дерева классификации** ... 22
   - 6.1. Misclassification error ... 23
   - 6.2. Gini Index (критерий Джини) ... 24
   - 6.3. Entropy (энтропия) ... 26
   - 6.4. Отличия в поведении Gini и энтропии ... 29
   - 6.5. Сквозной пример: самостоятельно собираем дерево классификации ... 31
7. **Особенности обучения деревьев** ... 33
   - 7.1. Алгоритмическая сложность ... 33
   - 7.2. Категориальные признаки ... 35
   - 7.3. Пропущенные значения ... 36
   - 7.4. Методы борьбы с пропущенными значениями MCAR/MAR ... 36
   - 7.5. Несбалансированные классы и цена ошибки ... 37
   - 7.6. Важности признаков (Feature importance) ... 39
   - 7.7. Три отличия permutation importance от MDI: ... 41
   - 7.8. Глубина дерева и переобучение ... 41
   - 7.9. Нестабильность деревьев ... 42
8. **Применение метода решающих деревьев** ... 43
   - 8.1. Преимущества ... 43
   - 8.2. Недостатки ... 44
9. **Деревья vs линейные модели** ... 44
10. **Если кратко** ... 48

Деревья решений

Контекст

Мы уже прошли линейные модели – основу курса машинного обучения. Деревья решений – следующий этап, где мы собираем реальные данные и строим модели, выходящие за пределы линейных границ. Изучив тему деревьев, ты получишь базу для перехода к ансамблевым методам.

Применение

С деревьями решений можно строить модели, которые объясняют свои выводы правилом «Если... то...». Это особенно ценно, когда нужна простая интерпретация решений. Алгоритм помогает в оценке кредитного риска, медицинской диагностике, маркетинговой сегментации, поиске брака на производстве. Например, банк может мгновенно отклонять подозрительные операции по цепочке вопросов: «Сумма > 500 долларов? Клиент за границей?» — а врач может классифицировать опухоль как доброкачественную или злокачественную по набору клинических показателей.

Краткий план

- → Что такое деревья решений
- → Виды деревьев решений
- → Технические аспекты дерева решений
- → Обучение дерева решений
- → Обучение дерева регрессии
- → Обучение дерева классификации
- → Особенности обучения деревьев
- → Применение метода решающих деревьев
- → Деревья vs линейные модели
- → Если кратко

1. Что такое деревья решений

Рассмотрим новый класс моделей — деревья решений. У них есть несколько значимых преимуществ.

- Высокая интерпретируемость результатов: можно легко проследить, как модель приняла решение.
- Сверхгибкость: можно создавать нелинейные зависимости любой сложности.

Сверхгибкость — следствие непараметричности подхода. Мы не фиксируем заранее количество параметров для обучения, модель сама адаптируется под данные, улавливая самые разные зависимости. В результате количество степеней свободы (параметры, которые подстраиваются под данные) в деревьях может достигать огромных значений.

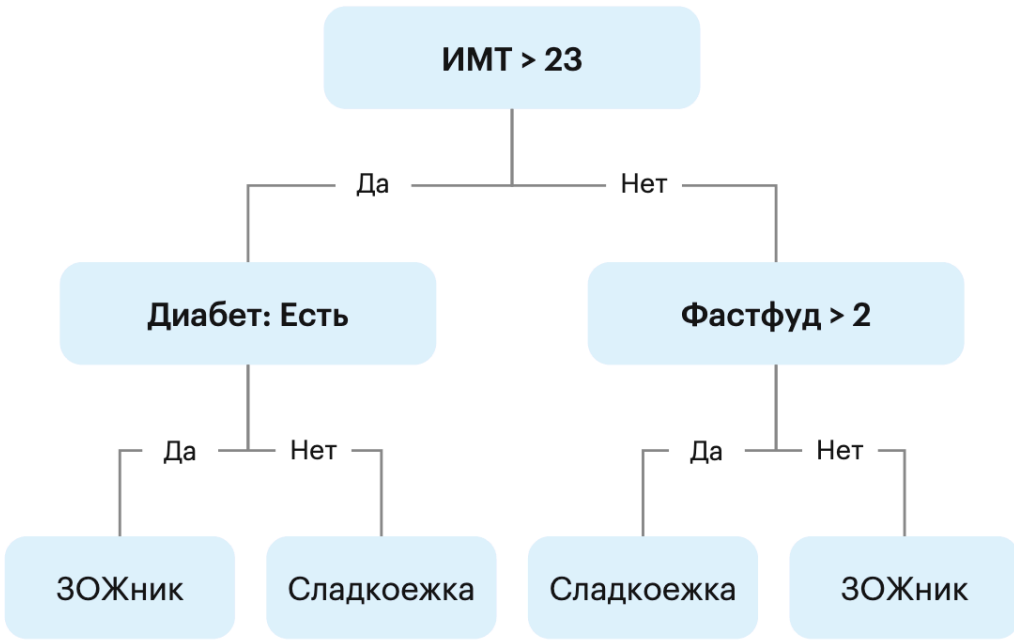
ВАЖНО

Дерево решений (англ. decision tree) — это алгоритм машинного обучения, который помогает принимать решения, задавая последовательность вопросов.

ПРИМЕР

К врачу приходит пациент с болями в желудке. Нужно провести осмотр и выдвинуть первую гипотезу о причине болей. Очевидно, что боль в желудке может быть из-за неправильного питания, поэтому сначала врач проверит данные пациента.

По показателям прежних пациентов и их образу жизни можно построить бинарное решающее дерево.



Рассмотрим компоненты деревьев решений на основе примера.

1. **Внутренние вершины, или узлы (англ. internal nodes)**, разбивают пространство признаков на части (так называемые **регионы**) согласно порогам. Каждый узел — это условие на каком-либо признаке. Среди узлов можно выделить **корень (англ. root)** — первую (верхнюю) внутреннюю вершину дерева.
   Внутренние вершины включают следующие условия.
   - *ИМТ* > 23 (корень), где ИМТ (индекс массы тела) принимает количественные значения (вещественные числа).
   - *Диабет*: есть. Диабет — категориальный признак, принимает значения из множества («Есть», «Отсутствует»).
   - *Фастфуд* > 2, где фастфуд — количественный признак, принимает значения из множества натуральных чисел и означает количество приёмов вредной пищи в неделю.
2. **Терминальные вершины, или листья (англ. terminal node/leaf)**, не делятся дальше и хранят итоговое предсказание для данного набора признаков — класс (или вероятность классов) в задачах классификации и число в задачах регрессии.
   В примере четыре листа, определяющих образ жизни пациентов по классам целевой переменной: зожник, сладкоежка. Так как целевая переменная принимает категориальные значения, то это задача классификации, а дерево в примере — дерево классификации.
3. **Дуги, или рёбра (англ. branch/edge)**, связывают родительскую и дочернюю вершины. Соответствуют одному исходу: «Да» или «Нет».

Главные достоинства модели — интерпретируемость и простота в понимании результатов. Дерево решений — способ принятия решений по алгоритму «Если… то…». У человека есть «лишние килограммы»? Если да, то есть ли у него диабет? Если есть, вероятно, он следит за питанием (зожник), потому что диабет ограничивает сладкое. А если нет, то, скорее всего, он любит поесть сладкого или вредного.

Получается, если пациент относится к разряду сладкоежек, то первой гипотезой врача о причине болей в желудке станут гастрит или язвенная болезнь.

В реальной жизни факторов, от которых зависит образ питания, больше: это количество углеводов/белков/жиров, объём и время приёмов пищи. Но даже такое небольшое дерево решений поможет врачу с постановкой диагноза.

1.1. ГРАФИЧЕСКОЕ ПРЕДСТАВЛЕНИЕ

А теперь поговорим о визуальном представлении деревьев. Дерево решений можно представить с помощью древовидной структуры, где дерево растёт сверху вниз, как в примере выше. Но одно и то же дерево можно представить разными способами, например как на рисунке 1.

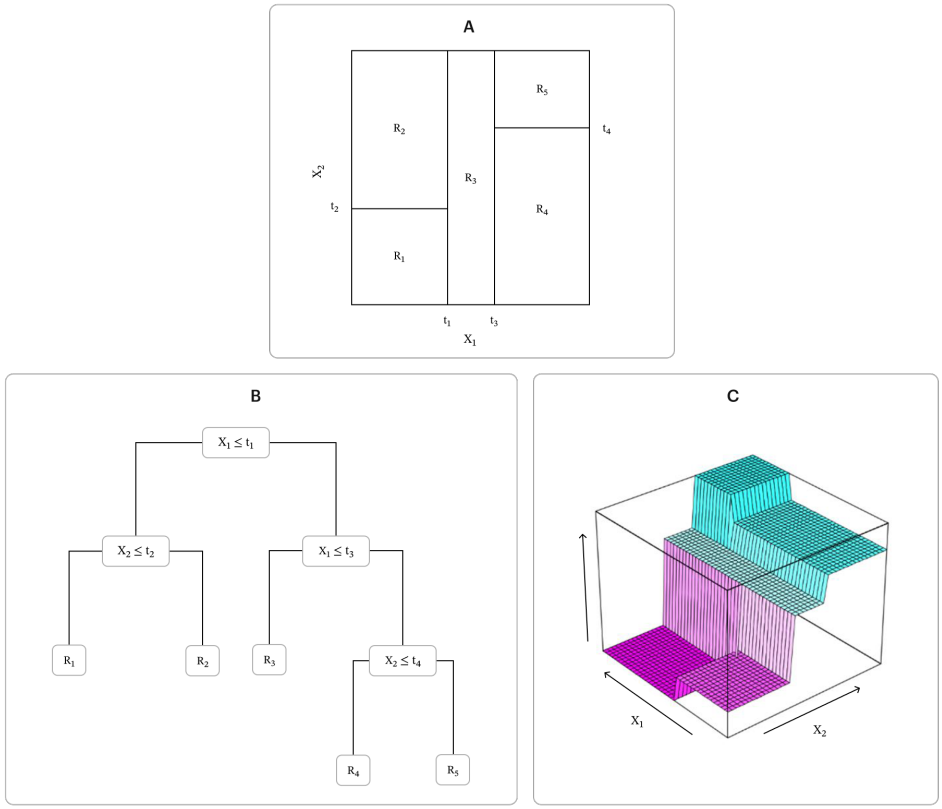


Рис. 1. Три способа представления одного и того же дерева регрессии, обученного по принципу CART. Источник: ESL, T. Hastie & R. Tibshirani, 2023

На рисунке 1 все изображения относятся к одному дереву, которое обрабатывает два непрерывных признака X_1 и X_2 и разбивает всё пространство признаков на части (регионы, R_1, R_2, R_3, R_4, R_5). Каждому отдельному региону присваивается значение непрерывного целевого признака Y.

Изображение 1B относится к древовидному представлению. Последовательно разбиваем два признака по конкретным условиям, в итоге приходим к пяти терминальным вершинам (или регионам) — вещественным числам (так как дерево регрессионное) из множества значений целевой переменной Y. Эти пять регионов (и присваиваемые им значения Y) — предсказания дерева. То же самое дерево можно представить в виде разбиения пространства признаков на регионы (изображение 1A). И снова каждому из регионов присваивается определённая оценка Y.

Эта оценка хорошо представлена на изображении 1C. Это то же самое разбиение, но уже в виде двумерной функции $X \to \hat{f}(X)$, где X — вектор признаков X_1 и X_2, а $\hat{f}(X)$ — оценка зависимости между X и Y. Рисунки 1A и 1C показывают одно и то же, но рисунок 1C отображает значения наших предсказаний для каждого региона: R_1, R_2, R_3, R_4, R_5, то есть показывает предсказания Y для каждого значения X_1 и X_2. Этот график можно сравнить с двумерным графиком предсказаний модели линейной регрессии, где плоскость предсказаний линейная, а не ступенчатая. Уже по этому графику видно, что деревья решений — это более гибкий алгоритм по сравнению с линейными моделями.

2. Виды деревьев решений

В decision-tree-литературе принято говорить о семействе методов, которые можно разложить по нескольким независимым осям. Ниже — самые базовые классификации деревьев. Комбинируя их, легко описать почти любой древесный алгоритм от классических до более современных моделей.

Тип целевой переменной	Дерево классификации	Предсказывает метку класса или вероятность классов
	Дерево регрессии	Предсказывает вещественную непрерывную величину
Бинарность разбиения узлов	Бинарное разбиение	Каждый узел делится на два подузла
	Мультисплит	Узлы делятся больше чем на два подузла
По форме разделяющей поверхности	Параллельно осям координат	Официальное название — осевое разбиение
	По линейной (и нелинейной) комбинации признаков	При таком разбиении существует высокий риск сильного переобучения дерева
По стратегии построения	Top-down, сверху вниз	Построение от корня к листьям
	Bottom-up, снизу вверх	Построение от листьев к корню
По обработке данных	Batch training	Дерево обучается сразу на всех данных
	Online training	Дерево постепенно дообучается на новых данных

Критериев для классификации деревьев больше, выше указаны самые базовые. На основании этих критериев и отличаются друг от друга разные алгоритмы деревьев решений: например, алгоритм C4.5 поддерживает и вещественные, и категориальные признаки, тогда как алгоритм ID3 работает только с категориальными переменными.

Один из стандартных и распространённых алгоритмов деревьев решений — это **CART**, **Classification and Regression Trees**.

ВАЖНО

CART

CART применяется в задачах и регрессии, и классификации. Это простой для понимания алгоритм, который широко используется в реальных задачах.

Его особенности:

- бинарные, осевые и жадные разбиения узлов (см. ниже);
- построение дерева от корня к листьям;
- минимальные требования к данным;
- нечувствительность к масштабу;
- допускает пропуски в данных (это упрощает препроцессинг данных в реальных задачах).

CART — алгоритм золотой середины, и в рамках этого лонгрида мы будем в основном работать именно с ним.

Основные признаки дерева, построенного по методу CART:

- последовательное разбиение пространства признаков. Разбиваем один общий регион на два региона, далее разбиваем один из двух получившихся регионов ещё на два региона, далее разбиваем один из трёх регионов ещё на два региона и так далее. Этот способ разбиения называется **жадным (greedy)**, рассмотрим его далее;
- **бинарное** разбиение узлов дерева, когда каждый узел разбивается на два подузла;
- разбиение пространства признаков одномерно: каждое условие разбиения внутренних вершин зависит только от одной переменной: $X_1 <= t_1, X_2 <= t_2, ...$ В таком случае разбиение называется **осевым**, деление пространства на регионы происходит параллельно осям координат (см. рисунок 1А). Но если бы разбиение, например, происходило по типу $X_1 X_2 <= t_1$, то оно бы считалось многомерным и у регионов на рисунке 1А была бы непрямоугольная форма.

Эти три условия построения дерева формируют принцип **рекурсивного бинарного разбиения (РБР)** — обучения по принципу CART.

Однако не все деревья обучаются согласно РБР (как, например, дерево на рисунке 2).

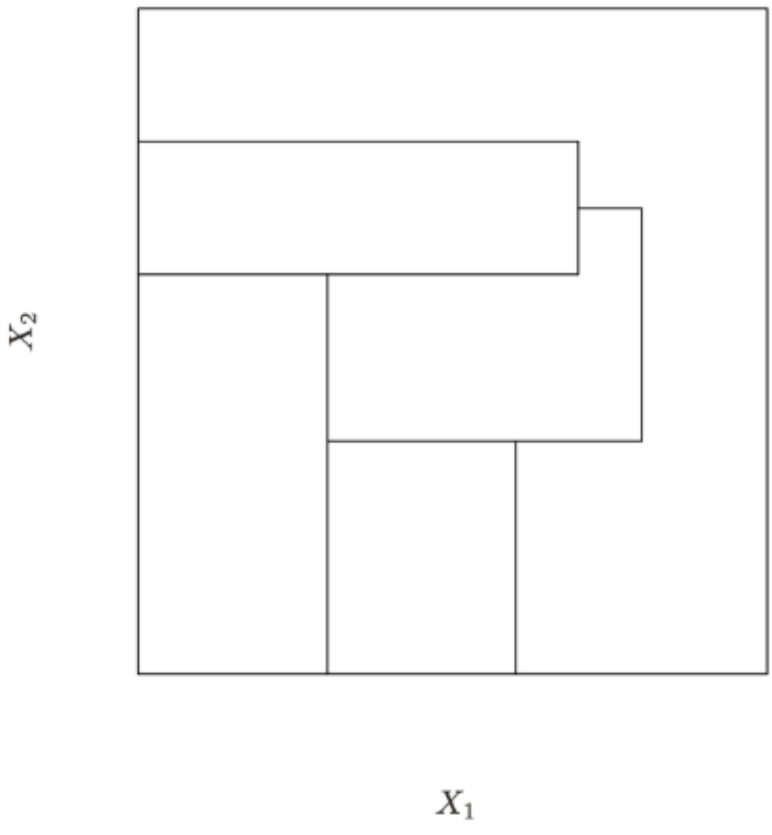


Рис. 2. Такое разбиение на прямоугольные регионы нельзя получить рекурсивным бинарным осевым разбиением, поэтому это не CART

Далее в лонгриде мы будем исследовать в основном CART-деревья, обученные по принципу РБР.

ВАЖНО

Бинарная древовидная CART-структура точно описывается условной инструкцией if-else, которая встречается во многих языках программирования.

Вот пример на языке Python (на основе схемы про образ жизни пациентов).

```python
def predict_diet(bmi, diabetes, junkfood_number):
    """
    Очень простое «дерево» в виде вложенных if-else.
    Возвращает название образа жизни и диеты.
    """
    # Узел 1 (корень)
    if bmi > 23:
        # Узел 2
        if diabetes = "yes":
            return "healthy style"        # Лист 1
        else:
            return "sweet tooth"          # Лист 2
    else:
        # Узел 3
        if junkfood_number > 2:
            return "sweet tooth"  # Лист 3
        else:
            return "healthy style"   # Лист 4
```

3. Технические аспекты дерева решений

В этой части:

- математически определим, что такое дерево решений;
- рассмотрим дерево с точки зрения древовидной структуры и определим, что происходит в каждой внутренней и терминальной вершинах;
- рассмотрим дерево с точки зрения разбиения пространства признаков на регионы

3.1. ОПРЕДЕЛЕНИЕ ДЕРЕВА РЕШЕНИЙ

Начнём с базового определения того, что такое граф.

Согласно теории графов, граф — это совокупность вершин и рёбер, допускающая наличие циклов, кратных и петлевых рёбер (см. рисунок 3). Дерево решений — это связный ациклический (без циклов) граф с n вершинами и ровно n-1 рёбрами.

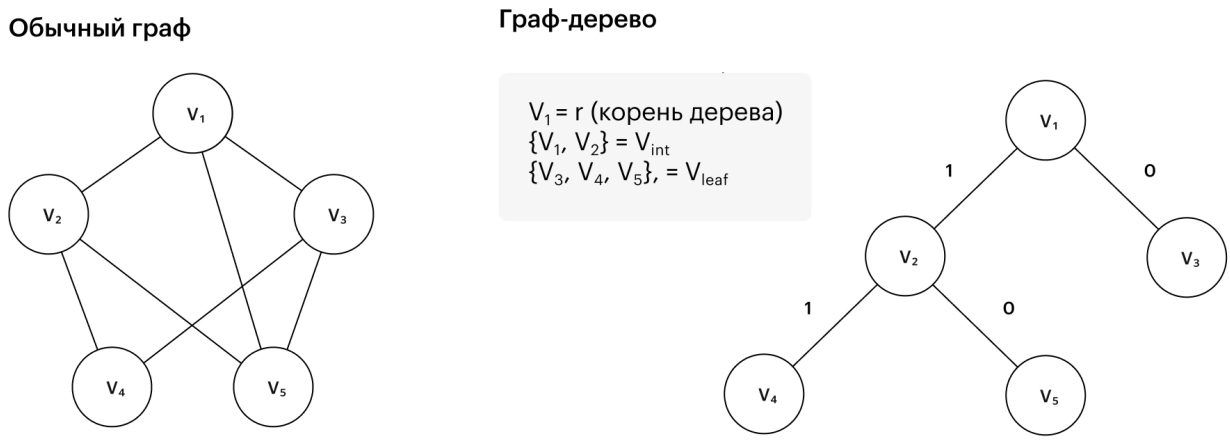


Рис. 3. Сравнение обычного графа с графом-деревом. В дереве указаны все внутренние и терминальные вершины, рёбра помечены значениями 1 и 0

Разберёмся чуть подробнее.

Пусть:

- $\mathcal{X}$ — пространство признаков (если все признаки вещественные, тогда $\mathcal{X} \subseteq \mathbb{R}^d$);
- $\mathcal{Y}$ — пространство ответов (для классификации $\mathcal{Y} = \{1, \dots, K\}$, для регрессии $\mathcal{Y} = \mathbb{R}$).

Дерево решений T, согласно теории графов, задаётся как упорядоченное ориентированное дерево $T = (V, E, r)$ с корнем $r \in V$, где E — множество рёбер между вершинами, а множество вершин V разбивается:

- на внутренние вершины V_{int} (решающие узлы);
- листья V_{leaf}.

3.2. ФУНКЦИИ В УЗЛАХ

В дереве каждой внутренней вершины $v \in V_{\text{int}}$ сопоставляется булева функция (англ. boolean function):

$$h_v : \mathcal{X} \to \{0, 1\}.$$

Если деление внутри вершины происходит по количественной переменной, тогда $h_v(x) = \mathbb{I}(x_{j_v} \leq t_v)$, где x — объект (наблюдение);

$\mathbb{I}(...)$ — индикаторная функция, которая возвращает 1, если соблюдается условие внутри круглых скобок, и 0, если условие не соблюдается;

x_{j_v} — количественный признак, по которому происходит деление;

$j_v \in \{1, \dots, d\}$ — индекс признака;

$t_v \in \mathbb{R}$ — порог.

Аналогично в случае, если деление происходит по категориальной переменной, функцией обычно будет $h_v(x) = \mathbb{I}(x_{j_v} \in C_v)$, где x_{j_v} — категориальный признак, по которому происходит деление;

$j_v \in \{1, \dots, d\}$ — индекс признака;

C_v — подмножество категорий признака x_{j_v}.

Рёбра из v помечены значениями 1 и 0 (рисунок 3). При обходе дерева объект x идёт налево, если $h_v(x) = 1$, и направо, если $h_v(x) = 0$.

Ниже показана работа булевой функции в узле.

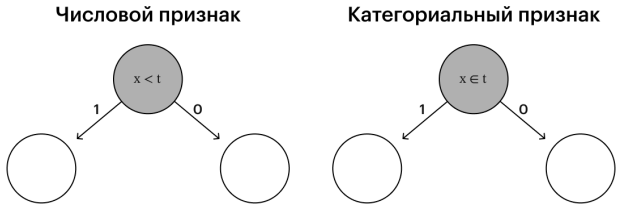


Рис. 4. Условия в узлах при делении вершины по числовому и категориальному признакам. Мы сейчас математическим образом описали всё то, что интуитивно разбирали в первой главе

ВАЖНО

Функции $h_v(x)$ применимы только в тех случаях, когда деление происходит по одному признаку. В таком случае эти функции называются одномерными предикатами, а деление называется осевым. Подробнее это условие мы разбирали выше.

Но деление пространства признаков на регионы может происходить сразу по нескольким признакам. Например, деление внутренней вершины происходит по условию «Если произведение первого и второго признаков больше нуля, тогда...». Такие случаи рассматривать пока не будем, так как они приводят к серьёзному переобучению (severe overfitting) модели и требуют очень аккуратного подхода.

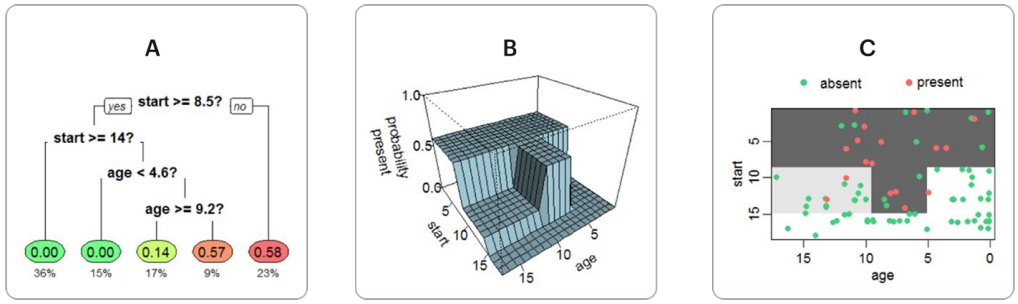


Рис. 5. Дерево классификации в разных представлениях. Источник: «Википедия»

На рисунке 5 – два признака и два класса целевой переменной. На рисунке 5A листья принимают значения вероятностей первого класса, а под листьями указано, какой процент наблюдений находится в данном регионе (листе). 5C показывает разбиение пространства на регионы: чем темнее регион, тем выше вероятность первого класса. Рисунок 5B показывает двумерную функцию оценки вероятностей первого класса для каждого региона.

3.3. МЕТКИ В ЛИСТЬЯХ

ВАЖНО

Каждому листу $\ell \in V_{\text{leaf}}$ назначается константное значение $c_\ell \in \mathcal{Y}$. В дереве регрессии c_ℓ — это значения из какого-то бесконечного множества чисел (обычно вещественных чисел), а в дереве классификации — значения из конечного множества меток (классов). Кстати, при классификации каждому листу могут также присваиваться вероятности каждого из классов (или одного класса, см. рисунок 5). В таком случае c_ℓ будет вектором вероятностей с размерностью в количество классов, которые может принимать целевая переменная.

Отметим интересное свойство деревьев: один родительский узел может разделиться на два дочерних узла (листа) с одинаковым предсказанием, особенно в задачах классификации. Такое происходит, чтобы разделить регион на два региона с «более уверенной» и «менее уверенной» отметкой, где в первом регионе вероятность предсказанной отметки значительно выше, чем во втором.

Пример этого свойства можем наблюдать на рисунке 6. Внутренние узлы (MaxHR < 161.5) и (Ca > 0.5) разбиваются на листья с одинаковым предсказанием. Очевидно, в таком случае у какого-то листа предсказание будет «более уверенным», а у другого — «менее уверенным».

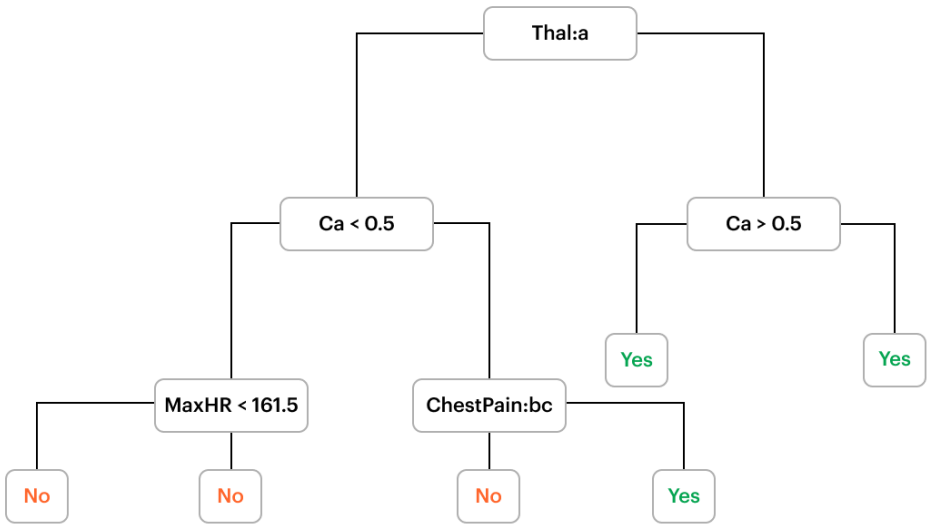


Рис. 6. Два дочерних листа дерева иногда могут нести одно предсказание.
Источник: ISL; Hastie, Tibshirani

3.4. РАЗБИЕНИЕ ПРОСТРАНСТВА ПРИЗНАКОВ

В деревьях мы сначала разбиваем целое пространство признаков на два региона, далее каждый из этих двух регионов последовательно делится на ещё два региона и так далее. По сути, мы совершаем одно и то же действие на всё меньших и меньших частях задачи — именно это и есть суть рекурсии.

ОПРЕДЕЛЕНИЕ

Рекурсивный алгоритм — это алгоритм, который для решения задачи вызывает сам себя с чуть более простой версией этой же задачи. Он повторяет эти вызовы до тех пор, пока не дойдёт до базового случая, который решается напрямую без дальнейшей рекурсии.

Таким образом, булевы функции h_v рекурсивно порождают семейство непересекающихся регионов $\{R_\ell\}_{\ell \in V_{\text{leaf}}} \subset \mathcal{X}$ (гиперпрямоугольников при осевых разбиениях), причём:

$$\mathcal{X} = \bigsqcup_{\ell \in V_{\text{leaf}}} R_\ell.$$

Значит, объединение всех получившихся регионов и формирует пространство признаков. На рисунке 1A, а также на рисунке 5C указаны простые примеры такого разбиения.

3.5. ПРЕДСКАЗАТЕЛЬНАЯ ФУНКЦИЯ ДЕРЕВА

Зная, что дерево задаётся $T = (V, E, r)$, определим отображение:

$$f_T : \mathcal{X} \longrightarrow \mathcal{Y}, \qquad f_T(x) = \sum_{\ell \in V_{\text{leaf}}} c_\ell \, \mathbb{I}(x \in R_\ell).$$

Кусочно-постоянная функция f_T возвращает метку того листа, в который попал объект x при нисходящем обходе от корня. Функция f_T будет многомерным обобщением рисунка 5B, а также той самой функции из рисунка 1C, о которой мы говорили.

ВАЖНО

В случае с деревом классификации с двумя возможными классами c_ℓ может принимать значения вероятности одного из классов в каждом листе дерева $\ell \in V_{\text{leaf}}$ (см. рисунок 5).

Если проводить параллель с линейной регрессией, наша обученная ступенчатая функция f_T выполняет ту же роль, что и обученная регрессия $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \ldots$. Таким образом, в линейной регрессии, имея входные данные по какому-то наблюдению, можно подставить эти данные в обученную регрессию («подставить значения X») и получить оценку целевой переменной $\hat{y}$. Также в алгоритме деревьев, имея данные по наблюдению, можно посмотреть, в какой регион в пространстве признаков попадает данное наблюдение. И из этого оценить значение Y, которое может принимать это наблюдение (подставить X в $f_T(x)$ и посчитать значение функции).

Цель древесного алгоритма достигнута: функция предсказывает значения Y, полагаясь на значения X.

3.6. ВЫВОДЫ

- Дерево решений можно представить как ориентированный ациклический граф с разделением вершин на внутренние узлы и листья.
- Последовательное применение осевых булевых функций (предикатов) h_v по числовым порогам или подмножествам категорий задаёт рекурсивное разбиение пространства признаков на непересекающиеся регионы R_ℓ.
- Дерево — это кусочно-постоянная функция f_T, которая в каждом регионе возвращает константу: число (регрессия) или метку/вектор вероятностей (классификация).
- При осевых сплитах регионы — гиперпрямоугольники, а вся поверхность предсказаний ступенчатая.

4. Обучение дерева решений

В этой части:

- посмотрим, как работает обученное дерево и как с помощью дерева решений делать предсказания;
- проанализируем, как обучать деревья и чем отличаются деревья регрессий от деревьев классификаций;
- изучим тонкости не только CART, но и более продвинутых методов построения деревьев.

Обучение дерева решений на данных выборки — это процесс максимизации функционала качества (или минимизации функционала ошибки) для поиска оптимального набора параметров.

ОПРЕДЕЛЕНИЕ

Параметры в дереве решений — это:

- **признаки**, по которым происходит деление региона пространства признаков на две части. Или признаки, по которым булева функция внутри каждой внутренней вершины делит эту вершину на две подвершины;
- **пороги**, по которым эти признаки делятся на две части;
- **предсказания** в каждом из листьев.

Наша задача — минимизировать функционал ошибки, разбив пространства признаков на прямоугольные регионы и присвоив каждому региону одно значение целевой переменной.

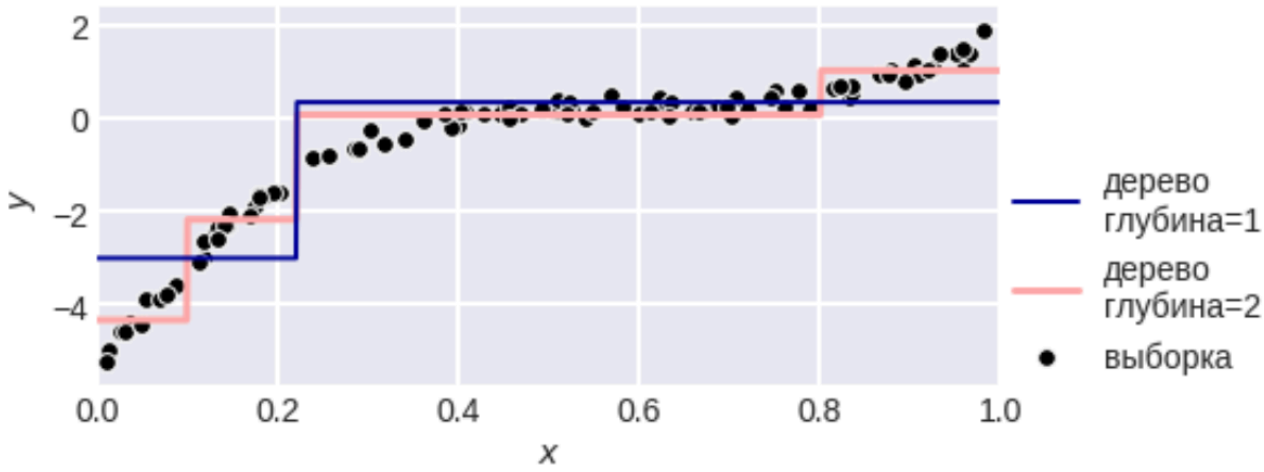


Рис. 7. Деревья регрессии: функции предсказаний целевой переменной y по данным о признаке x

На рисунке 7 два дерева регрессий с глубиной 1 и 2. Первое дерево разбивает признак x в значении 0.22. Всем объектам, у которых $x < 0.22$, предсказывает значение $\hat{y} = -2.9$, а всем остальным объектам присваивает $\hat{y} = 0.2$.

Второе дерево сначала разбивает признак x в том же значении 0.22 и потом каждый из двух получившихся регионов снова разбивает в точках 0.1 и 0.8. После чего каждому из четырёх регионов (листьев) присваивает по одному константному значению $\hat{y}$. Таким образом, можно использовать эти обученные деревья для предсказания $\hat{y}$ на новых данных, получив новый объект $x = 0.5$. Предсказания значения целевого признака у этого объекта по этим двум деревьям будут $\hat{y} = 0.2$ и $\hat{y} = 0$. Похожий пример

на рисунке 8.

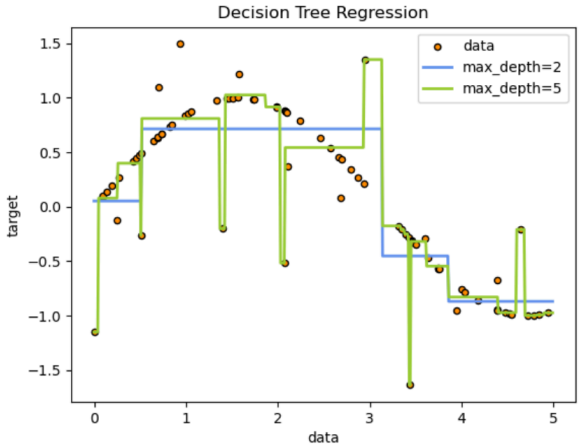


Рис. 8. Ещё одно дерево регрессии. Источник: scikit-learn

Древесная модель — это **непараметрический алгоритм**. Параметры в этой модели заранее не фиксируются, и их количество может достигать больших значений (больше данных → больше параметров). Действительно, так и получается: чем больше данных, тем больше будет разбиений пространства признаков и тем больше значений $\hat{y}$ придётся присваивать каждому региону.

Как подобрать такие сплиты (или разбиения) переменных и обучить оптимальное дерево с хорошей предсказательной способностью на новых данных?

Во-первых, можно подобрать такие сплиты, которые минимизировали бы loss на обучающих данных, как, например, в линейной регрессии, когда минимизировали MSE. Но есть проблема: дерево решений, в отличие от линейной регрессии, — это непараметрическая модель, а значит, дерево в результате обучения может оказаться неограниченно гибким. Если цель — минимизировать обучающую ошибку, то дерево может обучиться до такой степени, когда каждое конкретное наблюдение будет в отдельном регионе (количество листьев = количество наблюдений). Хотя такая стратегия и приведёт к нулевой обучающей ошибке, она покажет высокую тестовую ошибку. Итог: слабая способность предсказывать значения на новых данных.

Во-вторых, можно попробовать минимизировать функцию потерь модели при каком-то условии. Например, целиться на такое разбиение пространства признаков, при котором loss минимизируется для какого-то заранее зафиксированного количества листьев дерева. Для этого подбираем такое дерево, которое даёт минимальный loss среди всех деревьев с десятью листьями. В таком случае, если мы имеем дело с деревом регрессии и определяем loss как квадратичную ошибку, нашей задачей окажется:

$$\min_{R_1, R_2, \ldots, R_J} \sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 \qquad \text{for } J = 10,$$

где

$$\hat{y}_{R_j} = \text{ave}(y_i \mid x_i \in R_j).$$

Если у каждого региона на пространстве признаков есть одно предсказанное значение $\hat{y}_{R_j}$, то общая квадратичная ошибка будет минимизироваться на всех данных, только если $\hat{y}_{R_j}$ — среднее арифмети-

ческое целевой переменной в каждом регионе.

ВАЖНО

Теоретически такой наивный метод обучения дерева хорош, но его крайне трудно реализовать. Из-за своей дискретной структуры и непостоянного количества параметров деревья решений нельзя продифференцировать по параметрам и найти с помощью градиентного спуска оптимум (как у линейной регрессии).

Только представь, что для каждого разбиения первого признака надо смоделировать разбиение второго признака, для каждого разбиения первого и второго признаков надо смоделировать разбиение третьего признака и так далее. Даже в случаях с категориальными фичами (уже не говоря о количественных) это быстро приведёт к комбинаторному взрыву — экстремальному росту количества возможных исходов, которые нужно сравнивать.

Поиск такого дерева — NP-полная задача. Человечество пока не знает, как решить её за полиномиальное время.

Из-за сложностей поиска глобального минимума функционала ошибки рассмотрим два неидеальных, но хороших на практике метода.

- Применить жадный алгоритм (greedy algorithm). Жадность заключается в максимизации метрики (минимизации лосса) на каждом сплите. То есть на каждом разбиении узла дерева наша задача — пошагово максимизировать качество модели.
- Оптимизировать наивный метод обучения, который можно ускорить и асимптотически, и в k (константу) раз.

ПРИМЕР

В рамках жадного рекурсивного бинарного разбиения наша задача — на каждом разбиении дерева регрессии/классификации максимизировать функционал качества:

$$Q(R, j, t) = H(R) - \left(\frac{|R_\ell|}{|R|}H(R_\ell) + \frac{|R_r|}{|R|}H(R_r)\right)$$

подобрав оптимальные параметры на текущем сплите:

$$(j, t) = \arg max_{j,t}\, Q(R, j, t).$$

Обозначения в верхних двух выражениях:

- R — выбранный для следующего сплита регион пространства признаков;
- j — индекс признака, по которому происходит деление региона;
- t — порог, по которому происходит деление признака;
- $Q(R, j, t)$ — функционал качества (метрика);
- $H(\bullet)$ — мера неопределённости (impurity measure) (далее разберём подробнее);
- R_ℓ — левый регион после разделения родительского региона, аналогично с правым регионом;
- $|\bullet|$ — количество наблюдений в конкретном регионе.

Мера неопределённости региона (функционал ошибки) показывает, насколько объекты внутри региона «не похожи друг на друга»:

$$H(R) = \min_{c \in Y} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} L(y_i, c),$$

где c — наше константное предсказание, которое может быть вещественным числом (для ответов регрессии и меток класса) или вектором вероятностей всех классов внутри региона (только в задачах классификации).

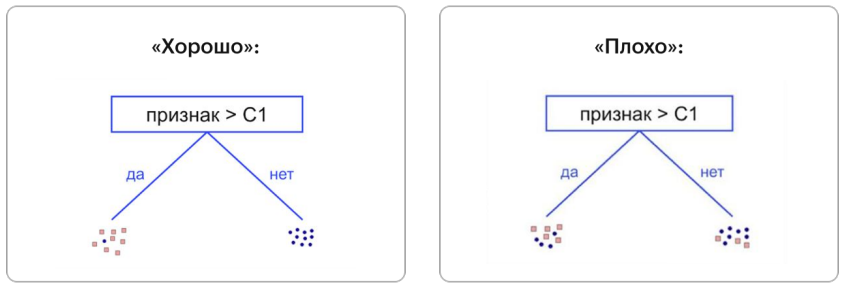


Рис. 9. Разбиение пространства с меньшей неопределённостью (слева) и большей неопределённостью (справа).

В случае с проблемой классификации неопределённость максимизируется, если внутри региона количество объектов каждого класса равно, и минимизируется, если внутри региона — объекты только одного из классов (см. рисунок 9). Если есть проблема регрессии, неопределённость увеличивается с ростом разброса (например, дисперсии) объектов. Разные меры неопределённости для регрессий и классификаций рассмотрим ниже.

Таким образом:

$H(R)$ — мера неопределённости в родительском регионе;

$\frac{|R_\ell|}{|R|} H(R_\ell) + \frac{|R_r|}{|R|} H(R_r)$ — общая взвешенная (нормированная) мера неопределённости в двух получившихся после разбиения регионах.

4.1. НОРМИРОВКА НЕОПРЕДЕЛЁННОСТЕЙ

Разберём, зачем нужна нормировка неопределённостей. Если просто считаем сумму $H(R_\ell) + H(R_r)$, то вес дочерних регионов одинаковый. В таком случае наш критерий начнёт переоценивать «фейковые» сплиты, которые делают один из листов крошечным, но абсолютно чистым. Разберём пример, чтобы понять, какой сплит лучше.

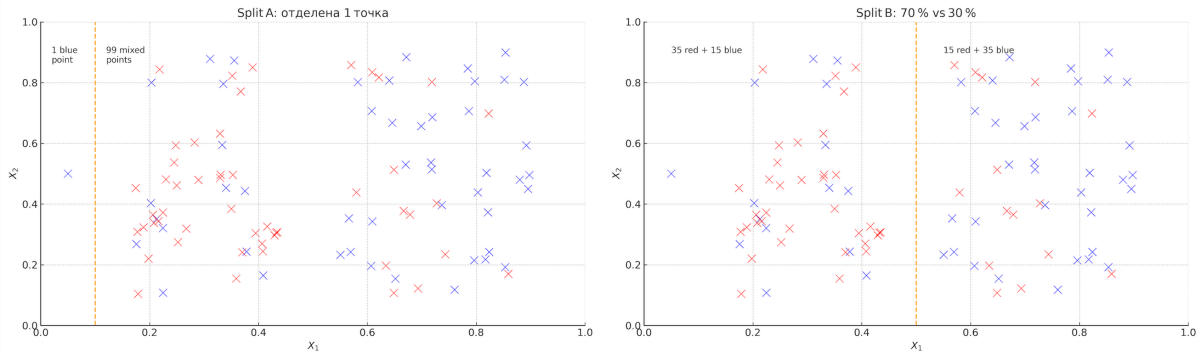


Рис. 10. Смысл применения нормировки в задаче оптимизации функции качества

ПРИМЕР

Задача бинарной классификации: надо поделить узел дерева на два подузла. В родительском узле — 100 объектов, соотношение классов — 50/50 (красный vs синий класс).

Сплит A: влево уходит один идеально чистый объект (соотношение классов — 100/0), вправо — 99 объектов с тем же разбиением ≈ 50/50.

Сплит B: делит выборку пополам, в каждом подузле соотношение классов объектов 70/30 и 30/70 соответственно.

Взглянем на рисунок 10.

Интуитивно понятно, что мера неопределённости чистого (левого) листа из одного объекта в сплите а равна нулю. А неопределённость у правого листа из того же сплита больше, чем неопределённость обоих листов из сплита B, так как разбиение 50/50 более зашумлённое, чем 70/30 (или 30/70).

Предположим, что мы всё-таки посчитали меры неопределённостей для обоих сплитов и получили $H(R_\ell) = 0$ и $H(R_r) = 0.5$ в случае сплита а и $H(R_\ell) = H(R_r) = 0.42$ в случае сплита B (далее разберём, как это рассчитывается).

Без нормировки сравниваем $H(R_\ell) + H(R_r)$ для обоих сплитов:

- а даёт $0 + 0.5 = 0.5$;
- B даёт $0.42 + 0.42 = 0.84$.

Алгоритм «подумал бы», что А круче (меньше мера неопределённости). А теперь попробуем применить нормировку:

- а $\rightarrow 0.01 \cdot 0 + 0.99 \cdot 0.5 = 0.495$;
- B $\rightarrow 0.5 \cdot 0.42 + 0.5 \cdot 0.42 = 0.42$.

Теперь выигрывает действительно полезный сплит B. Так что нормировка спасает нас от ловушки «чистого одноэлементного листика» и делает разные сплиты сопоставимыми по масштабу.

Итак, максимизация функционала качества Q происходит при увеличении разности между критерием неопределённости до разбиения региона и после.

Наша задача — подобрать такой сплит региона, который уменьшит меру неопределённости модели на этом шаге и точнее сгруппирует данные на два получившихся региона.

Параллельно с этим для каждого региона подбираем константное предсказание. Обычно это среднее арифметическое значений целевой переменной по всем наблюдениям внутри региона $c = \text{ave}(y_i \mid x_i \in R_j)$, так как именно среднее по выборке минимизирует сумму квадратов ошибок. Однако другие меры неопределённости предполагают другие статистики (например, медиану), а в задачах классификации обычно выбираются другие, нечисловые, характеристики.

Теперь понятно, что в первую очередь для каждого региона надо подобрать такой сплит, который будет минимизировать разброс внутри этого региона.

4.2. СТРУКТУРА ПОСТРОЕНИЯ ДЕРЕВА

Разберём, как выглядит пошаговая структура построения дерева и как выбрать следующий регион для «сплита». Для этого есть несколько стратегий.

1. Рост дерева в глубину (сверху вниз): разбиваем корень на два узла, далее берём один из узлов (пусть левый) и разбиваем его на два узла, далее берём один из двух конечных узлов (пусть снова левый) и разбиваем на два узла. Так до тех пор, пока получившиеся регионы уже не получится разбивать дальше и они станут листьями. Далее идём по дереву «наверх» и раскрываем ближайшую вершину, которая не была раскрыта. Раскрываем её до тех пор, пока она не превратится в лист. И так далее.
   В теории графов это называется обходом графов в глубину (DFS). Также этот подход используется во многих классических методах построения деревьев решений (например, в CART и C4.5).
2. Рост дерева в ширину (слева направо): разбиваем корень на два узла, далее разбиваем левый узел на два узла и правый узел на два узла, получаем четыре узла. Далее раскрываем каждый из этих четырёх узлов ещё на два узла. Идём слева направо (см. рисунок 11).
   Такой подход называется **level-wise tree growth** (рост дерева по уровням), а в теории графов — обходом графов в ширину (BFS).

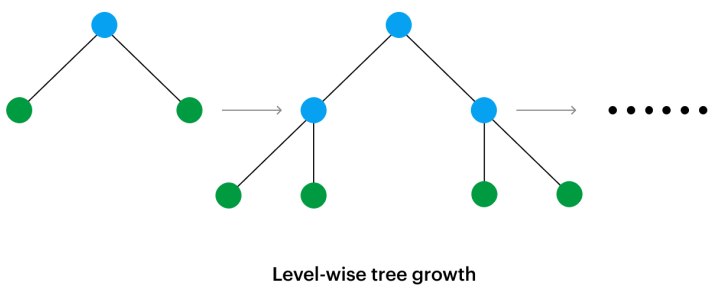
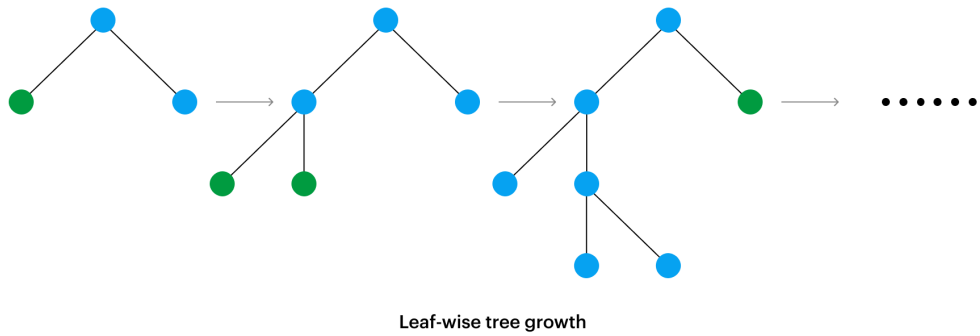


Рис. 11. Рост дерева в ширину. Источник: datascience.stackexchange

3. Рост дерева по наилучшим узлам: на каждом шаге выбирается регион, разбиение которого даст наибольший выигрыш глобально. В отличие от двух методов выше, здесь алгоритм смотрит на все листья, доступные для разбиения, и выбирает среди них тот, где потенциальное улучшение качества максимальное. Таким образом, дерево растёт не строго по уровням или глубине, а в порядке убывания информационного выигрыша (см. рисунок 12).
   Этот метод также называется **leaf-wise tree growth** (рост дерева по лучшим листам), и он широко используется в бустинговых методах.



Leaf-wise tree growth

Рис. 12. Рост дерева по наилучшим узлам. На каждом шаге максимизируется общее качество модели.
Источник: datascience.stackexchange

ВАЖНО

В случае leaf-wise growth нашей задачей на каждом шаге будет максимизация $(R, j, t) = \arg max_{R,j,t}\ Q_{best}(R, j, t)$, где $Q_{best}(R, j, t) = Q(R, j, t) * |R|$. Здесь множитель $|R|$ будет играть роль нормализации. Так как теперь выбираем лучший сплит среди всех регионов, прирост качества в каждом из регионов нормализуем по количеству наблюдений в этом регионе. Например, если прирост качества при разбиении первого региона — 10, а прирост качества при разбиении второго — 5, но количество наблюдений в первом регионе в 20 раз меньше, чем во втором, то прирост качества при разбиении второго региона окажется более ценным для обучения модели.

Все три метода последовательного построения дерева приводят к одинаковым результатам, если цель — построить полное и максимально глубокое дерево. Но дерево решений — алгоритм, который очень легко переобучается на существующих данных. При построении полного дерева есть риск, что дерево будет ловить «шум» в данных: переобучится на обучающей выборке и утратит обобщающую способность.

Избежать этого можно методами, которые ограничивают гибкость модели. Например, стоп-критерии или ранняя остановка обучения дерева по заданным критериям (остановить рост дерева, если количество листьев превысило 20). Для разных критериев остановки будет иметь значение, какой из трёх методов построения дерева был выбран. Далее отдельно рассмотрим процесс обучения для деревьев регрессии и классификации.

5. Обучение дерева регрессии

Вернёмся к анализу функционала качества Q и меры неопределённости H (будем называть её impurity). Наша задача — минимизировать impurity → максимизировать качество на каждом сплите региона. Но определение impurity (усреднённого функционала ошибки) напрямую зависит от типа задачи: для регрессии лоссы будут определяться иначе, чем для классификации. Начнём с разбора мер неопределённости в задачах регрессии.

Мера impurity напрямую зависит от функционала ошибок:

$$H(R) = \min_{c \in Y} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} L(y_i, c).$$

Самый применяемый функционал ошибки в задачах регрессии — квадратичная ошибка:

$$L(y_i, c) = (y_i - c)^2.$$

В таком случае неопределённость внутри региона будет измеряться среднеквадратичной ошибкой (Mean Squared Error, MSE):

$$H(R) = \min_{c \in Y} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} (y_i - c)^2.$$

ОПРЕДЕЛЕНИЕ

Среднее арифметическое — статистика, которая минимизирует MSE:

$$H(R) = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} (y_i - \frac{1}{|R|} \sum_{(x_j, y_j) \in R} y_j)^2 = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} (y_i - \bar{y})^2.$$

Таким образом, мера неопределённости при квадратичной ошибке — это дисперсия, которая и служит показателем разброса данных внутри региона. Получается красивая картинка: оценка значения в каждом листе — это среднее, а выбирать сплиты надо так, чтобы сумма дисперсий в листьях была как можно меньше.

Однако квадратичный функционал ошибки не единственный, который применяют для поиска оптимальных сплитов.

Рассмотрим другой loss, например абсолютную ошибку:

$$L(y_i, c) = |y_i - c|.$$

ОПРЕДЕЛЕНИЕ

И тогда задача — минимизировать среднеабсолютное отклонение (Mean Absolute Error, MAE) от медианы:

$$H(R) = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} |y_i - MEDIAN(Y)|,$$

так как именно медиана значений внутри региона минимизирует абсолютную неопределённость.

Разберём, как построить дерево регрессии в Python. Для этого даже не придётся писать дерево с нуля (хотя в одном из семинаров попробуем), ведь есть библиотека scikit-learn с алгоритмами обучения деревьев. Осталось только подобрать гиперпараметры.

```python
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(criterion='squared_error', # критерий расщепления
    # «squared_error»/«absolute_error»
     splitter='best',                # разбиение «best»/«random»
     max_depth=None,                 # допустимая глубина
     min_samples_split=2,            # минимальная выборка для разбиения
     min_samples_leaf=1,             # минимальная мощность листа
     min_weight_fraction_leaf=0.0,   # аналогично с весом
     max_features=None,              # признаков для нахождения разбиения
     random_state=None,
     max_leaf_nodes=None,            # допустимое число листьев
     min_impurity_decrease=0.0,      # порог изменения «зашумлённости»
     ccp_alpha=0.0)                  # для подрезки

model.fit(X, y)
```

Такие гиперпараметры, как `max_depth`, `min_samples_split`, `min_samples_leaf` и другие, нужны для предотвращения переобучения дерева.

ВАЖНО

Примечание. sklearn.tree использует нормированный функционал качества каждого сплита: $Q(R, j, t) = \left(H(R) - \left(\frac{|R_\ell|}{|R|}H(R_\ell) + \frac{|R_r|}{|R|}H(R_r)\right)\right) * \frac{|R|}{N}$. Это может пригодиться при подборе гиперпараметра `min_impurity_decrease`.

6. Обучение дерева классификации

Наша задача — на каждом шаге подбирать такой сплит, который максимизирует разницу между зашумлённостью (неопределённостью) родительского региона и взвешенной суммой зашумлённостей двух дочерних узлов. В деревьях регрессии эта зашумлённость измерялась в мерах среднеквадратичной/среднеабсолютной ошибок, но это не подходит для задач классификации. Для таких задач нужны другие меры неопределённости. Посмотрим, как они выглядят.

Для начала определим пропорцию наблюдений класса k в каком-то регионе R:

$$\hat{p}_k = \frac{1}{|R|}\sum_{x_i \in R} \mathbb{I}(y_i = k).$$

Мера неопределённости формулируется так же:

$$H(R) = \min_{c \in Y} \frac{1}{|R|}\sum_{(x_i, y_i) \in R} L(y_i, c).$$

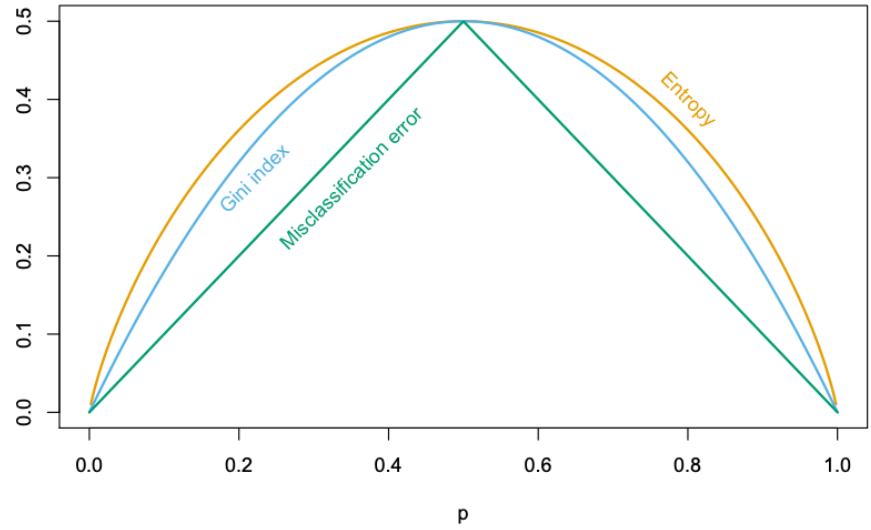


Рис. 13. Зависимость мер неопределённости от пропорции первого класса в регионе. Двухклассовая классификация. Энтропия нормализована. Источник: ESL; Hastie, Tibshirani

6.1. MISCLASSIFICATION ERROR

Определим функцию потерь как **0–1 loss function**:

$$L(y_i, c) = \mathbb{I}[y_i \neq c].$$

Этот loss присваивает единицу всем неправильным предсказаниям и ноль всем правильным.

Мерой неопределённости здесь станет:

$$H(R) = \min_{c \in Y} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} \mathbb{I}[y_i \neq c].$$

Здесь c – это конкретный класс предсказания. Очевидно, что предсказанием (классом), которое будет минимизировать неопределённость в этом случае, станет **мода** $c = k^* = \arg\max_k \hat{p}_k$:

$$H(R) = \frac{1}{|R|} \sum_{(x_i, y_i) \in R} \mathbb{I}[y_i \neq k^*] = 1 - p_{k^*}.$$

ОПРЕДЕЛЕНИЕ

Эта мера неопределённости называется долей ошибок классификации, или **misclassification error**.

Если определить функционал потерь как 0–1 loss, то минимизироваться мера неопределённости (средний loss) будет, если для региона классом предсказания станет часто встречающийся класс в этом регионе. Дерево классификации с misclassification error неопределённостью будет создавать такие регионы, которые максимизируют долю самого часто встречающегося класса внутри каждого региона. На рисунке 13 (двухклассовая классификация) видно, что misclassification error минимален в случаях, когда пропорция наблюдений одного класса минимизируется, а другого класса максимизируется. И наоборот, максимум зашумлённости достигается в точке $\hat{p}_1 = \hat{p}_2 = 0.5$.

Например, в случае с двумя классами, если выбор будет стоять между.

- Сплит 1. Первый регион (60 наблюдений первого класса, 40 наблюдений второго класса), второй регион (30 наблюдений первого класса, 40 наблюдений второго класса).
- Сплит 2. Первый регион (80 наблюдений первого класса, 20 наблюдений второго класса), второй регион (10 наблюдений первого класса, 60 наблюдений второго класса)

Тогда дерево с 0–1 loss примет решение в пользу второго сплита, так как именно второй сплит будет максимизировать вероятности самых частых классов.

Действительно, misclassification error создаёт наиболее «чистые» регионы с наименьшим количеством разнородности. Несмотря на свою простоту, эта мера зашумлённости редко используется на практике. Всё из-за того, что misclassification error фокусируется только навероятностях одного определённого класса, не принимая во внимание весь вектор вероятностей по всем классам.

6.2. GINI INDEX (КРИТЕРИЙ ДЖИНИ)

ПРИМЕР

Дерево выбирает между двумя сплитами.

- Сплит 1. Первый регион (300, 100), второй регион (100, 300) по количеству наблюдений.
- Сплит 2. Первый регион (200, 400), второй регион (200, 0) по количеству наблюдений.

Посчитаем взвешенный misclassification error для обоих сплитов:

$$\frac{|R_\ell|}{|R|}H(R_\ell) + \frac{|R_r|}{|R|}H(R_r) = \frac{400}{800} * (1 - \frac{300}{400}) + \frac{400}{800} * (1 - \frac{300}{400}) = 0.25;$$

$$\frac{|R_\ell|}{|R|}H(R_\ell) + \frac{|R_r|}{|R|}H(R_r) = \frac{600}{800} * (1 - \frac{400}{600}) + \frac{200}{800} * (1 - \frac{200}{200}) = 0.25.$$

Взвешенные misclassification errors для обоих сплитов равны, хотя второй сплит в среднем генерирует более «чистые» регионы. Один только регион (200, 0) уже говорит о, вероятно, хорошем разбиении узла. Эта мера зашумлённости недостаточно чувствительна ко всем анализируемым вероятностям.

Потому нужны другие меры неопределённости для задач классификации. Следующие две меры, которые мы разберём, основываются на суррогатных функциях потерь, аппроксимирующих 0–1 loss, разобранный выше.

Первая такая мера — критерий Джини. Эта мера неопределённости основывается на метрике Бриера (Brier score):

$$BS = \frac{1}{N}\sum_{t=1}^{N}(f_t - o_t)^2,$$

где f_t — вероятность предсказания какого-то класса у объекта;

o_t — фактическое значение целевой переменной объекта;

N — количество объектов.

Если, например, $f_t = 100\%$, а $o_t = 0$, то метрика Бриера достигает максимального значения, так как предсказание оказалось абсолютно неверным. Таким образом, метрика Бриера уменьшается при улучшении качества предсказания и может использоваться как функция потерь в структуре решающих деревьев:

$$H(R) = \min_{\sum_k c_k = 1} \frac{1}{|R|} \sum_{(x_i, y_i) \in R} \sum_{k=1}^{K} (c_k - \mathbb{I}[y_i = k])^2 .$$

Здесь, в отличие от случая с misclassification error, $(c_1, c_2, \ldots, c_k)$ вектор вероятностей классов — внутри региона. Оптимальное значение этой метрики достигается на векторе c, состоящем из выборочных оценок частот классов: $(\hat{p_1}, \ldots, \hat{p_k}), \quad \hat{p_k} = \frac{1}{|R|} \sum_i \mathbb{I}[y_i = k]$. Наши оценки вероятностей разных классов в регионе равны эмпирическим частотам этих классов в регионе. Например, если сейчас в регионе есть 80% наблюдений первого класса и 20% наблюдений второго класса, то все будущие наблюдения, попадающие в этот регион, будут иметь классовое распределение 80:20 (согласно нашей оценке).

ОПРЕДЕЛЕНИЕ

Подставив частоты классов $(\hat{p_1}, \ldots, \hat{p_k})$ на место вектора c, получаем критерий Джини:

$$H(R) = \sum_{k \neq k'} \hat{p}_k \hat{p}_{k'} = \sum_{k=1}^{K} \hat{p_k}(1 - \hat{p_k}).$$

Снова посмотрим на рисунок 13, где показываются значения зашумлённостей в двухклассовом случае. Критерий Джини — это дифференцируемая версия misclassification error, так как строится этот критерий на гладкой функции потерь (для этого и вводится эта суррогатная функция потерь). Таким образом, критерий Джини лучше подходит для задач оптимизации. Критерий Джини так же, как и misclassification error, поощряет сплиты, которые делят пространство признаков на более «чистые» регионы. Но, в отличие от misclassification error, этот критерий более чувствителен к изменениям в пропорциях классов в узлах дерева, а потому критерий Джини — один из двух наиболее часто применяемых методов оценки зашумлённости.

Итак, результатом обучения дерева классификации в конкретном листе может быть как предсказание одного класса (обычно моды), так и предсказание в виде вектора вероятностей (вектора частот). Если наша задача — дать конкретное предсказание в виде одного класса каждому листу, нам необязательно предсказывать моду. Например, можно предсказать класс k с вероятностью $\hat{p_k}$ (для этого применяем методы семплирования из категориальных распределений). В таком случае критерий Джини будет иметь интересное свойство: он будет равен мат. ожиданию misclassification error в узле.

Более того, если для каждого класса мы присвоим объектам этого класса значение 1, а объектам всех остальных классов значение 0, то в таком случае критерий Джини будет выступать в роли суммы дисперсий среди всех классов, так как дисперсия случайной величины Бернулли и есть $Var(X) = p(1 - p)$.

6.3. ENTROPY (ЭНТРОПИЯ)

Энтропия — это ещё один метод измерения неопределённости.

В теории информации энтропия случайной величины (или энтропия Шеннона) количественно определяет средний уровень неопределённости, связанной с потенциальными состояниями переменной или возможными результатами. Она измеряет ожидаемое количество информации для описания состояния переменной с учётом распределения вероятностей по всем возможным состояниям.

ОПРЕДЕЛЕНИЕ

Пусть X — дискретная случайная величина, которая может принимать значения x из множества $\mathcal{X}$ и распределена согласно функции вероятности $p : \mathcal{X} \to [0, 1]$. Тогда энтропия определяется

$$H(X) := -\sum_{x \in \mathcal{X}} p(x) \log p(x),$$

где логарифм обычно имеет основание 2 (двоичная энтропия, измеряемая в битах). С точки зрения статистики значение энтропии — это мат. ожидание $\mathbb{E}[-\log p(X)]$, которое ещё называют ожидаемым удивлением (expected surprisal).

Разберём, что такое «количество информации» в одном исходе:

- если событие почти достоверно ($p \to 1$) — наступление события не удивляет и почти не несёт информации;
- если событие редкое ($p \to 0$) оно, наоборот, очень информативно.

Эту «стоимость удивления» измеряет величина $-\log_2 p$ (англ. surprisal): чем меньше вероятность, тем больше бит нужно, чтобы закодировать исход. Энтропия — это среднее удивление по всем исходам.

ПРИМЕР

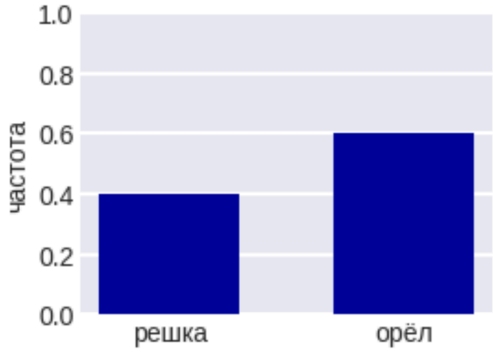
Рассмотрим бытовую аналогию — игру «угадай за 20 вопросов». Энтропия в битах приблизительно равна среднему числу вопросов с ответом «да/нет», нужных, чтобы точно определить исход:

- если классов два и они равновероятны — нужен один вопрос (энтропия = 1 бит);
- если один класс выпадает почти всегда — ответ известен заранее, вопросов почти не требуется (энтропия → 0).

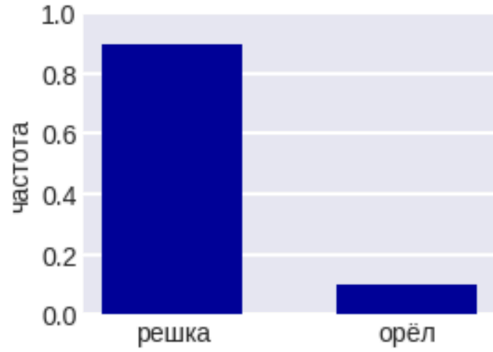
Поскольку $0 \leq p(x) \leq 1$, то каждое слагаемое $-p(x) \log_2 p(x) \geq 0$ — поэтому энтропия неотрицательна. Минимум $H = 0$ достигается при полной определённости (один исход с вероятностью 1). Максимум достигается при равномерном распределении и равен $\log_2 K$, где K — это число исходов:

- для двух классов — это $\log_2 2 = 1$ бит;
- для четырёх — это $\log_2 4 = 2$ бита и так далее.

Итог: чем более выровнены вероятности классов, тем выше энтропия.
И чем сильнее перекос к одному классу, тем она ниже.



$$-\frac{4}{10}\log_2\frac{4}{10}-\frac{6}{10}\log_2\frac{6}{10}\approx 0.97$$

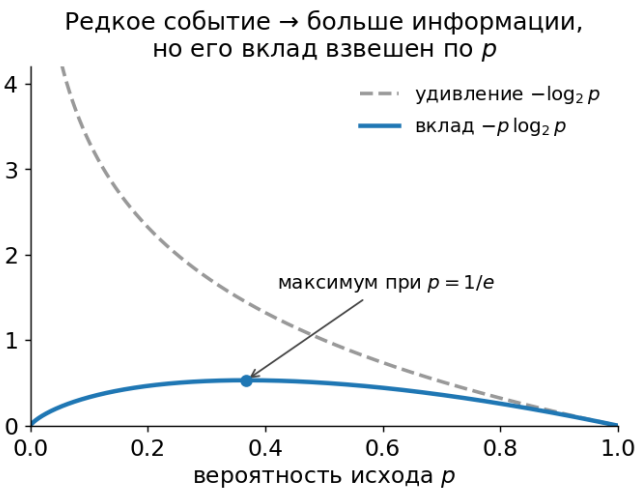


$$-\frac{9}{10}\log_2\frac{9}{10}-\frac{1}{10}\log_2\frac{1}{10}\approx 0.47$$

Рис. 14. Значения энтропии при броске двух смещённых монет

ПРИМЕР

На рисунке 14 — два эксперимента по подбрасыванию «искривлённой» монеты в воздух. В первом случае вероятность выпадения орла и решки одинакова, а потому значение энтропии близко к максимальному $0.97 \approx 1$. Во втором эксперименте монета смещена в сторону выпадения решки, в 90% случаев при подбрасывании такой монеты выпадает решка. Значение энтропии, ожидаемого удивления от исхода эксперимента, в этом случае значительно меньше. Все значения энтропии (нормированные, с максимумом в 0.5) в двухклассовой классификации указаны на рисунке 13.



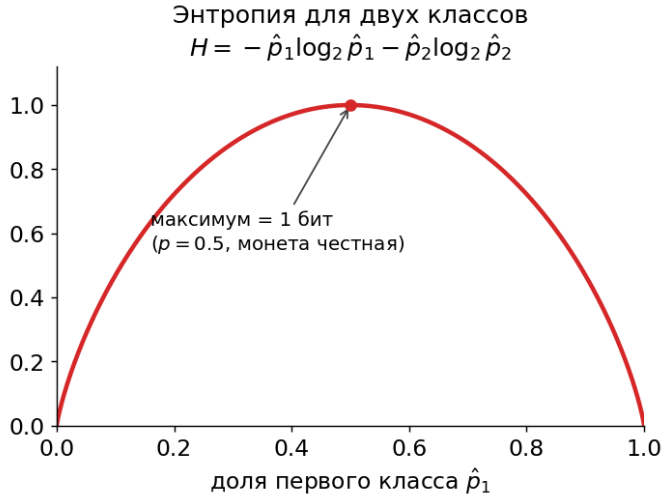


Рис. 15. Слева: удивление $-\log_2 p$ растёт для редких событий, но вклад исхода в энтропию $-p\cdot\log_2 p$ взвешен по его вероятности и обнуляется на обоих краях (максимум при p=1/e). Справа: энтропия двухклассового распределения

Левый график рисунка 15 объясняет, почему вклад одного исхода $-p\log_2 p$ обнуляется и при $p \to 0$, и при $p \to 1$. В первом случае исход бесконечно удивителен, но почти не встречается. Во втором — встречается часто, но совсем не удивляет. Получается, что максимум вклада приходится на «средние исходы» $p = 1/e \approx 0.37$. Сумма таких вкладов по всем классам и даёт энтропию.

Значение энтропии идеально вписывается в систему обучения деревьев классификации, ведь именно этот критерий измеряет неравномерность данных.

Выводим меру неопределённости из среднего значения функционала ошибки. Здесь за функцию потерь выступает лог-лосс (cross-entropy loss), отрицательное лог-правдоподобие (log-likelihood), и выводится из задачи максимизации лог-правдоподобия (= минимизации минус лог-правдоподобия):

$$P(y \mid x, c) = P(y \mid c) = \prod_{(x_i, y_i) \in R} P(y_i \mid c) = \prod_{(x_i, y_i) \in R} \prod_{k=1}^{K} c_k^{\mathbb{I}[y_i = k]},$$

$$H(R) = \min_{\sum_k c_k = 1} \left(-\frac{1}{|R|} \sum_{(x_i, y_i) \in R} \sum_{k=1}^{K} \mathbb{I}[y_i = k] \log c_k \right),$$

где c — это, как и в критерии Джини, вектор вероятностей классов в регионе.

Оптимизация в виде минимизации Лагранжиана приводит к тому, что минимизироваться значение минус лог-правдоподобия будет, когда вектор c соответствует частотам классов в листе $(\hat{p}_1, \dots, \hat{p}_k)$, как и в критерии Джини. Оба критерия аппроксимируют один и тот же 0–1 loss, так что это логично. Подставляя значения частот на место вектора c, получаем то самое выражение для энтропии:

$$H(R) = -\sum_{k=1}^{K} \hat{p}_k \log \hat{p}_k.$$

ВАЖНО

Интересно, что теория информации определяет прирост информации (information gain) как уменьшение энтропии в результате разбиения набора данных по оптимальному кандидату:

$$IG(Y \mid X) = H(Y) - H(Y \mid X).$$

Это значение показывает, насколько знание X уменьшило неопределённость в Y. Это отсылает нас к критерию качества Q, который определяется по тому же принципу.

На практике значения критериев Джини и энтропии для каждого сплита дерева очень схожи и часто приводят к обучению одного и того же дерева (но не всегда).

6.4. ОТЛИЧИЯ В ПОВЕДЕНИИ GINI И ЭНТРОПИИ

Обе меры аппроксимируют один и тот же 0–1 loss, равны нулю на чистых узлах и максимальны при равных долях классов — отсюда и похожие формулы. Различаются они лишь формой кривой — энтропия сильнее выгибается вверх, чем Gini (рисунок 16).

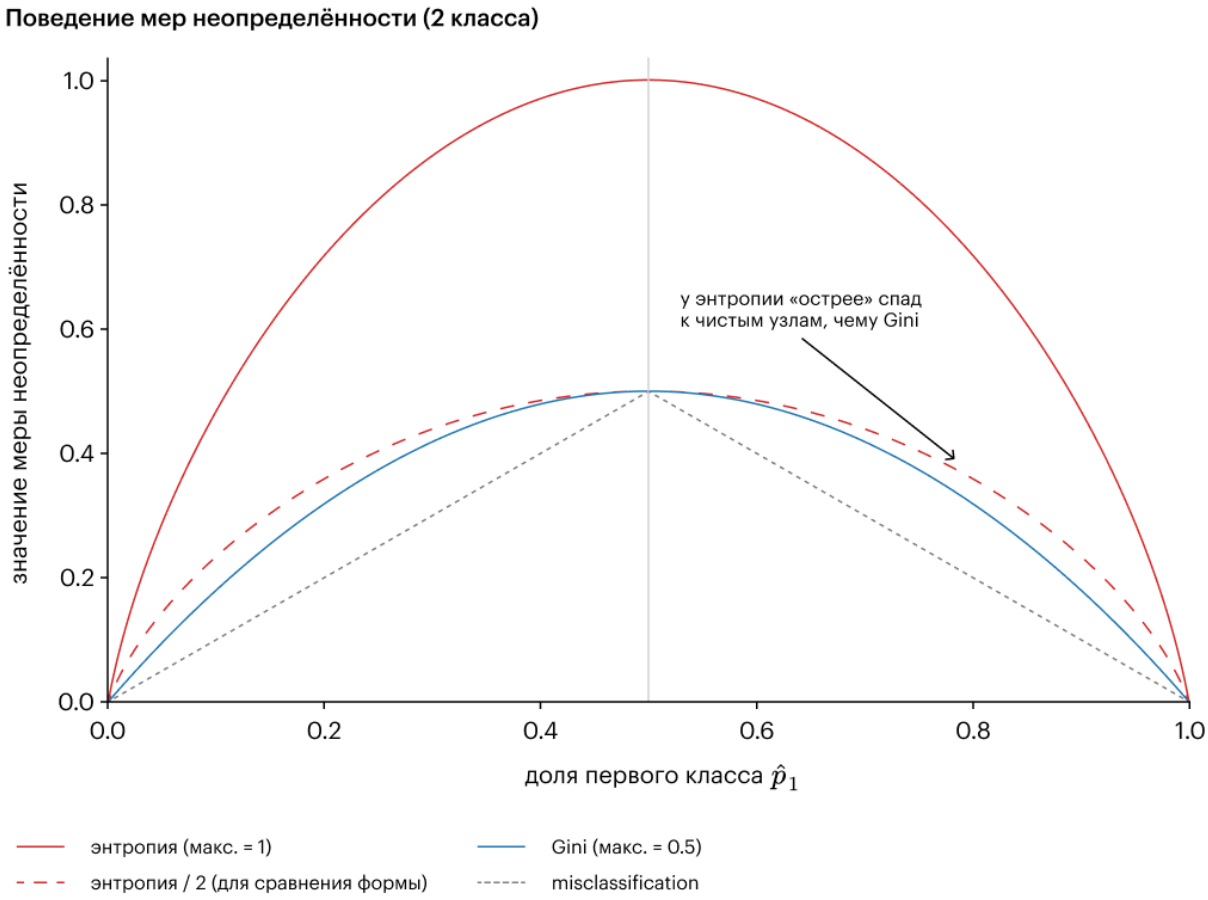


Рис. 16. Сравнение мер неопределённости в двухклассовом случае. Кривые Gini и нормированной энтропии почти совпадают по форме; энтропия чуть круче «штрафует» отклонение от чистого узла

Проанализируем графики на рисунке 16.

- **Масштаб.** Для двух классов максимум Gini доходит до 0.5, а энтропия в битах — до 1. Это просто разные шкалы, и на выбор сплита они не влияют.
- **Чувствительность.** Энтропия использует логарифм и поэтому сильнее реагирует на малые примеси других классов: вблизи чистого узла величина $-p \log p$ убывает круче, чем $p(1-p)$. Из-за этого энтропия чуть охотнее «дочищает» уже почти чистые узлы, а Gini — чуть охотнее выделяет самый крупный класс. В итоговых деревьях эта разница обычно почти незаметна.
- **Скорость.** Gini считается без логарифма, поэтому работает быстрее. Это одна из причин, почему именно он стоит критерием по умолчанию в scikit-learn.
- **Связь с теорией информации.** Энтропийный критерий — это в точности information gain $IG = H(Y) - H(Y \mid X)$ из формулы выше. У Gini такой трактовки нет, зато есть вероятностная: он равен ожидаемой ошибке «случайного» классификатора, который предсказывает класс k с вероятностью $\hat{p}_k$ (то есть вытягивает ответ из того же распределения долей классов, что и в узле). Действительно,

у случайно взятого объекта истинный класс равен k с вероятностью $\hat{p_k}$, и такой классификатор угадает его тоже лишь с вероятностью $\hat{p_k}$ — значит, ошибётся с вероятностью $1 - \hat{p_k}$. Усреднив по всем классам, получаем $\sum_k \hat{p_k}(1 - \hat{p_k}) = 1 - \sum_k \hat{p_k}^2$ — ровно критерий Джини.

Вывод. Выбор между Gini и энтропией почти никогда не критичен. Если нужна скорость — стоит использовать Gini, если же нужен язык «прироста информации» — применять энтропию.

- Сплит 1. Первый регион (300, 100), второй регион (100, 300) по количеству наблюдений.
- Сплит 2. Первый регион (200, 400), второй регион (200, 0) по количеству наблюдений.

Тогда как misclassification error для обоих сплитов равен, критерии Джини и энтропии показывают меньшие значения для второго сплита. Это говорит о более высокой чувствительности к вероятностям разных классов у критериев Джини и энтропии.

Рассмотрим, как в Python с библиотекой scikit-learn моделировать деревья классификации.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(criterion='gini', # критерий расщепления
                                                 # «gini»/«entropy»
    splitter='best',                # разбиение «best»/«random»
    max_depth=None,                 # допустимая глубина
    min_samples_split=2,            # минимальная выборка для разбиения
    min_samples_leaf=1,             # минимальная мощность листа
    min_weight_fraction_leaf=0.0,   # аналогично с весом
    max_features=None,              # признаков для нахождения разбиения
    random_state=3,
    max_leaf_nodes=None,            # допустимое число листьев
    min_impurity_decrease=0.0,      # порог изменения «зашумлённости»
    class_weight=None,              # веса классов («balanced» или словарь)
    ccp_alpha=0.0)                  # для подрезки

model.fit(X, y)
```

На этом завершаем разбор разных техник по обучению деревьев регрессий и классификаций. Отметим главное.

При обучении дерева решений есть два режима:

- наивный — перебирает все комбинации сплитов и теоретически ищет глобальный максимум;
- жадный — строит дерево рекурсивно, в каждой вершине выбирая лучший локальный разрез.

В практике почти всегда используем жадный из-за экспоненциальной стоимости полного перебора. Качество любого сплита оцениваем функционалом Q, который измеряет, насколько уменьшается неопределённость/ошибка после деления узла. Для регрессии неопределённость обычно меряют среднеквадратичной (MSE) или среднеабсолютной (MAE) ошибкой, а для классификации — misclassification error, индексом Джини или энтропией.

Сплит с максимальным выигрышем по Q выбирается, и процесс повторяется, пока не нарушится какой-то стоп-критерий (минимальное число объектов в листе, максимальная глубина). Само дерево можно порождать разными путями (например, «вглубь» или «вширь»); при использовании стоп-критериев разные методы построения могут выдавать отличающиеся итоговые деревья.

Итак, жадный поиск + правильная мера неопределённости дают нам практичный алгоритм, который шаг за шагом превращает исходные данные в иерархию простых правил. Все жадные методы редко приводят к теоретически оптимальным деревьям. Однако простота использования и высокая скорость обучения делают эти методы широко применимыми.

6.5. СКВОЗНОЙ ПРИМЕР: САМОСТОЯТЕЛЬНО СОБИРАЕМ ДЕРЕВО КЛАССИФИКАЦИИ

Соберём всё изученное в один проход. Для этого, в духе нашего примера про питание:

1. возьмём крошечную выборку из 12 пациентов с двумя признаками: «фастфуд в неделю» и «сладкие напитки в день»;
2. построим дерево самостоятельно ровно так, как это делает CART, — на каждом шаге перебираем все пороги по всем признакам и берём сплит с максимальным выигрышем Q по критерию Gini.

№	Фастфуд в неделю	Сладкие напитки в день	Класс
1	0	0	Зожник
2	1	0	Зожник
3	1	1	Зожник
4	2	0	Зожник
5	2	1	Зожник
6	1	2	Сладкоежка
7	2	3	Сладкоежка
8	3	0	Сладкоежка
9	3	2	Сладкоежка
10	4	1	Сладкоежка
11	5	0	Сладкоежка
12	6	2	Сладкоежка

Неопределённость корня. В корне 12 объектов — 5 зожников и 7 сладкоежек. А доли классов $\hat{p}_{\text{зож}} = 5/12$ и $\hat{p}_{\text{слад}} = 7/12$.

Посчитаем критерий Gini:

$$H(R_0) = 1 - \left(\left(\frac{5}{12}\right)^2 + \left(\frac{7}{12}\right)^2\right) = 1 - (0.174 + 0.340) = 0.486.$$

Перебор кандидатов в корне. Пороги-кандидаты — это середины между соседними значениями признака. Для каждого считаем взвешенную зашумлённость детей и выигрыш $Q = H(R_0) - \left(\frac{|R_\ell|}{|R|}H(R_\ell) + \frac{|R_r|}{|R|}H(R_r)\right)$.

Сплит	Левый (зожник, сладкоежка)	Правый (зожник, сладкоежка)	Взвешенный Gini	Q
Фастфуд ≤ 1.5	(3, 1)	(2, 6)	0.375	0.111
Фастфуд ≤ 2.5	**(5, 2)**	**(0, 5)**	**0.238**	**0.248**
Фастфуд ≤ 3.5	(5, 4)	(0, 3)	0.370	0.116
Сладкие ≤ 0.5	(3, 2)	(2, 5)	0.438	0.048
Сладкие ≤ 1.5	(5, 3)	(0, 4)	0.313	0.174
Сладкие ≤ 2.5	(5, 6)	(0, 1)	0.455	0.032

Разберём арифметику победителя — «Фастфуд ≤ 2.5». Слева оказываются 7 объектов (5 зож, 2 слад), а справа — 5 (0 зож, 5 слад):

$$H(R_\ell) = 1 - \left(\left(\frac{5}{7}\right)^2 + \left(\frac{2}{7}\right)^2\right) = 0.408, \qquad H(R_r) = 0 \text{ (чистый)},$$

$$Q = 0.486 - \left(\frac{7}{12} \cdot 0.408 + \frac{5}{12} \cdot 0\right) = 0.486 - 0.238 = 0.248.$$

Здесь $Q = 0.248$ — это максимальный выигрыш. У остальных порогов он меньше, поэтому и выбирается корневой сплит «Фастфуд ≤ 2.5».

- Правый регион (фастфуд > 2.5) сразу чистый: пять сладкоежек, $H = 0$, дальше делить нечего — это лист «сладкоежка».
- Левый регион в 7 объектов, $H = 0.408$ — ещё смешанный, поэтому рекурсивно делим уже его.

Рекурсия в левый узел. Переберём пороги заново, но только среди этих семи объектов — 5 зожников, 2 сладкоежки.

Сплит	Левый (зожник, сладкоежка)	Правый (зожник, сладкоежка)	Q
Фастфуд ≤ 0.5	(1, 0)	(4, 2)	0.027
Фастфуд ≤ 1.5	(3, 1)	(2, 1)	0.003
Сладкие ≤ 0.5	(3, 0)	(2, 2)	0.122
Сладкие ≤ 1.5	**(5, 0)**	**(0, 2)**	**0.408**
Сладкие ≤ 2.5	(5, 1)	(0, 1)	0.170

В этом случае побеждает сплит «Сладкие напитки ≤ 1.5». Он разбивает узел на два идеально чистых листа:

- 5 зожников (сладких напитков ≤ 1);
- 2 сладкоежки (≥ 2).

При этом выигрыш $Q = 0.408$ максимален, оба ребёнка чистые, а рекурсия останавливается.

Итоговое дерево. Получилось дерево глубины 2 с тремя листьями.

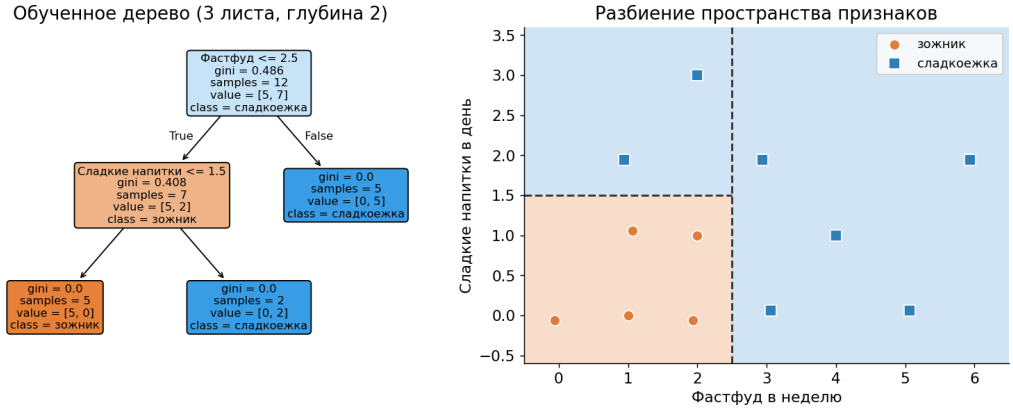


Рис. 17. Сквозной пример: слева — обученное дерево, справа — то же решение как разбиение пространства признаков. Граница ступенчатая: вертикальный разрез — по фастфуду, и горизонтальный — по сладким напиткам только в левой половине

Слева — само дерево, справа — то же решение, но показанное в виде разбиения пространства на прямоугольные регионы (вспомни рисунок 1А). Разберём, что представляет это дерево.

- Именно такое дерево и построит `DecisionTreeClassifier(criterion='gini')`: глубина 2, три листа и 100%-ная точность на обучающей выборке.
- Если повторить тот же перебор с энтропией вместо Gini, порядок предпочтений сплитов сохраняется и дерево получается идентичным. Это наглядно показывает: Gini и энтропия на практике почти всегда «соглашаются» друг с другом. - Но 100%-ная точность на 12 точках — это тревожный сигнал. На такой крошечной выборке дерево легко «заучивает» данные. На реальных данных мы бы ограничили глубину и проверяли качество на отложенной выборке — разберём это далее, в разговоре о переобучении.

7. Особенности обучения деревьев

7.1. АЛГОРИТМИЧЕСКАЯ СЛОЖНОСТЬ

Мы выяснили, что жадное построение перебирает на каждом узле все пороги по всем признакам. Рассчитаем, во сколько это обходится. Пусть:

- n — число объектов;
- p — число признаков.

Стоимость одного узла. Чтобы найти лучший сплит в узле, где лежат m объектов, для каждого признака нужно:

1) отсортировать объекты по этому признаку — $O(m \log m)$;
2) пройти по отсортированному списку слева направо, инкрементально пересчитывая доли классов (или суммы для регрессии) и значение Q в каждом кандидате-пороге — это $O(m)$, ведь переход к следующему порогу сдвигает счётчики ровно на одно наблюдение.

Сортировка доминирует, поэтому на один признак уходит $O(m \log m)$, а на все p признаков — $O(p\, m \log m)$.

Стоимость одного уровня дерева. Узлы одного уровня делят выборку на непересекающиеся части. Поэтому суммарно на уровне — ровно n объектов. Значит, суммарная стоимость сортировок на уровне не превышает $O(p\, n \log n)$, так как $\sum_i m_i \log m_i \le \left(\sum_i m_i\right) \log n = n \log n$.

Стоимость всего дерева. Чтобы получить её, осталось умножить стоимость узла на число уровней — глубину. У сбалансированного дерева глубина $O(\log n)$. Если на каждом узле пересортировывать признаки заново, то выходит:

$$O\big(p\, n \log n\big) \times O(\log n) = O\big(p\, n \log^2 n\big).$$

Трюк с предсортировкой. На практике (и в scikit-learn) каждый признак сортируют один раз в самом начале за $O(p\, n \log n)$. а затем поддерживают порядок при разбиениях, не сортируя заново. Тогда работа на уровне линейна — $O(p\, n)$ и сложность обучения сбалансированного дерева сворачивается до:

$$O\big(n\, p \log n\big).$$

Именно эту оценку обычно и называют сложностью обучения CART.

Худший случай. Дерево вырождается в «цепочку», когда каждый сплит отделяет всего по одному объекту. Тогда глубина становится $O(n)$ и обучение деградирует до $O(n^2\, p)$. На практике так случается редко, а ограничения вроде `max_depth` и `min_samples_leaf` не дают дереву углубиться.

Предсказание. Чтобы классифицировать один объект, его прогоняют от корня до листа. Это стоит O(глубины), то есть $O(\log n)$ для сбалансированного дерева. При этом прогноз почти не зависит от числа признаков, поэтому он получается очень быстрым даже у глубоких деревьев.

Этап	Сбалансированное дерево	Худший случай
Обучение (с предсортировкой)	$O(n\, p \log n)$	$O(n^2\, p)$
Предсказание (1 объект)	$O(\log n)$	$O(n)$
Память (хранение дерева)	$O(n)$	$O(n)$

Обучение деревьев масштабируется почти линейно по числу объектов — с точностью до логарифма. И линейно по числу признаков. Это одна из причин, почему деревья и их ансамбли остаются главным рабочим инструментом для табличных данных.

7.2. КАТЕГОРИАЛЬНЫЕ ПРИЗНАКИ

Наша задача при обучении дерева — разделить численный и категориальный признаки так, чтобы максимизировать разницу между объектами в двух дочерних регионах. В случае с численными признаками всё очевидно: нужно найти числовой порог, который «разорвёт» признак на две части. При работе с номинальными категориальными переменными, которые не делятся по одному числовому порогу, используем две стратегии.

- Отказ от бинарного разбиения в пользу мультисплита: разбить родительский узел с категориальным признаком на k дочерних узлов, где k — количество значений категориального признака. Такой подход может показать достойные результаты, но только в случае, если k — небольшое число. Если же категориальный признак может принимать много значений и $k \gg 0$, это может привести к огромному количеству листьев в обученном дереве.
- Сохранение бинарного разбиения и сравнение функционалов качества для каждого возможного сплита меток на две группы. Например, если качественный признак принимает значения из множества (A, B, C), то нужно сравнить неопределённости сплитов ((A, B), (C)), ((A, C), (B)), ((B, C), (A)) и выбрать сплит с наименьшей общей неопределённостью. Однако, согласно числам Стирлинга второго рода, число разбиений k-элементного множества на 2 подмножества равно $\left\{ \begin{matrix} k \\ 2 \end{matrix} \right\} = 2^{k-1} - 1$, а это экспоненциально взрывающаяся последовательность с растущим k. Если, например, качественный признак принимает 100 меток, алгоритму придётся сравнить качество $\approx 2^{99}$ сплитов, что крайне ресурсозатратный процесс.

ВАЖНО

Хорошая новость: есть метод, который позволяет сохранить бинарное разбиение при оптимальной алгоритмической сложности, но только в задачах регрессии или двухклассовой классификации. Разберём этот метод.

В случае бинарной классификации отсортируем признак по категориям в порядке возрастания/убывания доли объектов первого класса в категории. Например, сначала идут метки признака с минимальным количеством объектов первого класса, далее — метки со всё большим количеством первого класса. После этот признак можно разбивать так же, как и числовой, используя какой-то порог. В случае с мерами зашумлённости Джини или энтропии этот «лайфхак» приводит к тем же сплитам, что и перебор всех $2^{k-1} - 1$ комбинаций разбиений.

В случае регрессии сортируем категории признака по возрастанию/убыванию среднего арифметического в каждой категории.

Категориальных признаков с большим количеством меток k следует избегать: из-за большого количества возможных разбиений алгоритм чаще будет предпочитать сплит по этим признакам, что может привести к серьёзному переобучению.

7.3. ПРОПУЩЕННЫЕ ЗНАЧЕНИЯ

Классические алгоритмы деревьев решений (например, CART) хороши в работе с пропущенными значениями в данных. Разберём типы пропущенных данных.

- **Missing completely at random (MCAR)** — пропуски в значениях признака полностью случайны и не зависят ни от наблюдаемых, ни от ненаблюдаемых величин (абсолютная независимость от данных). На практике встречается редко.

  ПРИМЕР Лаборатория. Партия проб испорчена из-за сбоя центрифуги, поэтому у случайного поднабора пациентов отсутствует показатель, и это никак не связано с их состоянием.

- **Missing at random (MAR)** — пропуски зависят только от полностью наблюдаемых переменных.

  ПРИМЕР Медицинский опрос. Мужчины реже заполняют анкету с вопросами о депрессии, но это не зависит от реального уровня депрессии у мужчин.

- **Missing not at random (MNAR)** — пропуски зависят от самого́ скрытого значения (или других ненаблюдаемых факторов).

  ПРИМЕР Доходы в анкете. Люди с очень высоким доходом чаще пропускают вопрос «Сколько вы зарабатываете?».

  ПРИМЕР Симптоматика. Пациенты с тяжёлой депрессией с большей вероятностью откажутся заполнять шкалу тяжести депрессии.

Все методы, которые мы предложим далее, работают в случаях MCAR/MAR. В случае MNAR применяют более сложные механизмы, которые всё равно не смогут с малыми потерями справиться с проблемой неслучайных пропущенных данных.

7.4. МЕТОДЫ БОРЬБЫ С ПРОПУЩЕННЫМИ ЗНАЧЕНИЯМИ MCAR/MAR

- **Удаление** данных об объекте с пропущенной информацией. Этот метод используем в случаях, когда пропущенных данных не так много. Желателен также MCAR-сценарий.
- **Импутация (подстановка)** данных. Например, вместо пропущенных значений по признаку подставим среднее/медиану по этому признаку. Однако обычная замена данных не учитывает факт неопределённости в этих заменённых данных. Чтобы учесть неопределённость, подставим несколько разных значений, обучим несколько моделей, после чего измерим нестабильность предсказаний.
- Если переменная категориальная, присвоим пропущенным значениям этой переменной новый класс missing. Особенно это актуально в ситуациях, когда пропуски имеют системный характер и несут определённую информацию. Кстати, это может быть одним из способов проверки сценария MCAR: если класс missing связан со значениями других признаков или целевой переменной, то наш сценарий точно не MCAR.
- В случае зависимых признаков можно смоделировать зависимость переменной с пропущенными данными от значений других признаков. Например, можно построить линейную регрессию, которая будет предсказывать значения признака с пропущенными данными, и таким образом оценить пропуски в признаке.

- У разных алгоритмов могут быть специальные способы борьбы с missing data.

ВАЖНО

Дополнительно рассмотрим методы, применяемые в классических деревьях решений. Один из таких методов при обучении дерева — оценивать качество сплита признака с пропусками без учёта объектов с пропущенной информацией по этому признаку. Если решаем разбивать этот признак, отправляем эти объекты и в левое, и в правое поддерево. При этом присваиваем им веса: $\frac{|R_l|}{|R|}$ и $\frac{|R_r|}{|R|}$ для объектов в левом и правом поддереве. Далее эти веса учитываем как коэффициенты при $L(y_i, c)$ в формуле impurity. При предсказании тестовых данных, если у объекта нет информации по признаку в узле, отправляем его и в левое, и в правое поддерево. В конце измеряем конечное предсказание этого объекта как $\hat{y} = \frac{|R_l|}{|R|}\hat{y}_l + \frac{|R_r|}{|R|}\hat{y}_r$, где $\hat{y}_l$ и $\hat{y}_r$ — предсказания листов, в которые попали эти объекты.

Этот метод не единственный для борьбы с missing data. В алгоритмах CART часто используются суррогатные расщепления (surrogate splits). Этот механизм предполагает разбивать признаки с пропусками не напрямую, а через использование суррогатных переменных. Подберём суррогатный признак, который сильнее всего связан (скоррелирован) с признаком с пропусками, и смоделируем сплиты по этому суррогатному признаку. Этот метод также работает только в случаях сильной зависимости между признаками. Желателен сценарий MCAR/MAR .

7.5. НЕСБАЛАНСИРОВАННЫЕ КЛАССЫ И ЦЕНА ОШИБКИ

При обучении деревьев классификации мы предполагали, что точность предсказания каждого класса имеет равную ценность: последствия от предсказания класса 1 классом 2 были равны последствиям от предсказания обратного.

В задачах классификации, когда ошибка распределена неравномерно и ошибиться в предсказании каких-то классов чревато более серьёзными проблемами, чем в предсказании других классов, следует ввести поправки в обучение и предсказание модели.

ПРИМЕР

У кардиолога есть данные пациентов (имя, пол, возраст) с информацией о том, был у каждого пациента сердечный приступ или нет. Задача — определить зависимость между независимыми признаками и целевой переменной («сердечный приступ»), чтобы использовать эти знания для предсказания риска сердечного приступа у других пациентов. При этом результат false positive (сердечный приступ предсказан, хотя его не будет) лучше, чем false negative (сердечный приступ не предсказан, но произойдёт).

Один из способов учесть неравноценность ошибок — введение матрицы ошибок **L** размерностью K x K, где $L_{kk'}$ — значение потери при классификации объекта класса k как класса k'. Правильные предсказания не штрафуются: $L_{kk'} = 0$ для $k = k'$. Эту матрицу используем для коррекции разбиений дерева во время обучения. Например, внедрив её в критерий Джини: $\sum_{k \neq k'} L_{kk'} \hat{p}_k \hat{p}_{k'}$. По такой же схеме можно изменить критерий энтропии. Но это работает только в случае $K > 2$, а в бинарной классификации при обучении просто даём больший вес тем примерам, где ошибка дороже. Конечное предсказание в терминальных вершинах также меняется с моды на $k^* = \arg\min_k \sum_\ell L_{\ell k} \hat{p}_\ell$.

Матрицы ошибок и потерь. В теме несбалансированных классов сталкиваются два похожих по названию, но совершенно разных объекта. Разберём их.

- **Матрица потерь** (от англ. loss/cost matrix **L**, выше) — задаётся до обучения и говорит модели, какие ошибки дороже. Это инструмент обучения.
- **Матрица ошибок** (от англ. confusion matrix) — считается после обучения по предсказаниям и показывает, сколько объектов каждого класса куда попало. Это инструмент оценки — и именно её в русскоязычной литературе чаще зовут «матрицей ошибок». Этот термин ранее уже встречался в курсе, поэтому вспомним ключевые моменты.

	Предсказание «+»	Предсказание «−»
Истинный «+»	TP	FN
Истинный «−»	FP	TN

По матрице ошибок считают точность (precision) и полноту (recall):

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}.$$

Как это связано с дисбалансом: на несбалансированных данных обычная accuracy обманчива. Если положительных всего 1%, то модель, всегда отвечающая «отрицательный», даёт 99% точности, но не ловит ни одного важного случая. Поэтому смотрят не на голую accuracy, а на recall, precision, F1 или ROC-AUC.

Как встроить «цену ошибки» в дерево: `class_weigh` и `sample_weight`. На практике вместо матрицы потерь чаще пользуются весами. Идея та же: редким или дорогим объектам дают больший вес w_i, и тогда во всех формулах неопределённости вместо «числа объектов» считается их суммарный вес. Из-за этого доля класса в регионе становится взвешенной:

$$\hat{p}_k = \frac{\sum_{i \in R} w_i \, \mathbb{I}(y_i = k)}{\sum_{i \in R} w_i},$$

а критерий Gini или энтропии далее считается уже по этим взвешенным долям. В итоге редкий класс с большими весами начинает «весить» в impurity столько же, сколько частый. И дерево перестаёт его игнорировать.

В этом случае задать веса можно тремя способами:

- `sample_weight` — вектор весов для каждого объекта. Это самый гибкий способ: через него можно закодировать даже произвольную матрицу потерь, домножив вес объекта на цену его ошибки;
- `class_weight={0: w0, 1: w1}` — задаёт вес сразу целому классу, а не каждому объекту вручную;
- `class_weight='balanced'` — берёт вес класса обратно пропорционально его частоте ($w_k = \frac{n}{K\, n_k}$), уравнивая вклад всех классов. Это первый и самый частый приём для несбалансированных данных.

```python
# редкий положительный класс важнее в 5 раз
model = DecisionTreeClassifier(class_weight={0: 1, 1: 5})

# или автоматически, обратно пропорционально частотам классов:
model = DecisionTreeClassifier(class_weight='balanced')
model.fit(X, y)            # веса учитываются прямо в формуле impurity
```

Матрица потерь, `class_weight` и `sample_weight` — это три способа сказать одно и то же: «эти ошибки или классы важнее». Разница в том, на каком уровне нужно расставить приоритеты:

- матрица потерь удобна, когда цена зависит от пары (истинный класс → предсказанный);
- `class_weight` — когда достаточно повысить важность целого класса;
- `sample_weight` — когда достаточно повысить важность отдельных объектов.

Здесь важно помнить про компромисс. Взвешивание смещает дерево в пользу редкого класса — растёт recall. Но платой обычно становится большее число ложных срабатываний — падает precision. Поэтому баланс подбирают под задачу — например, прямо как у кардиолога, для которого пропустить инфаркт (FN) куда опаснее ложной тревоги (FP).

7.6. ВАЖНОСТИ ПРИЗНАКОВ (FEATURE IMPORTANCE)

В деревьях решений модель обучается на множестве признаков $X_1, \ldots, X_p$, но не все они одинаково влияют на прогноз. Оценка важности признаков отвечает на два практических вопроса.

1. **Интерпретация.** Какие факторы действительно «двигали» решения модели?
2. **Инженерия.** Какие столбцы можно отобрать, упростить, собрать ещё данных или, наоборот, убрать без потери качества?

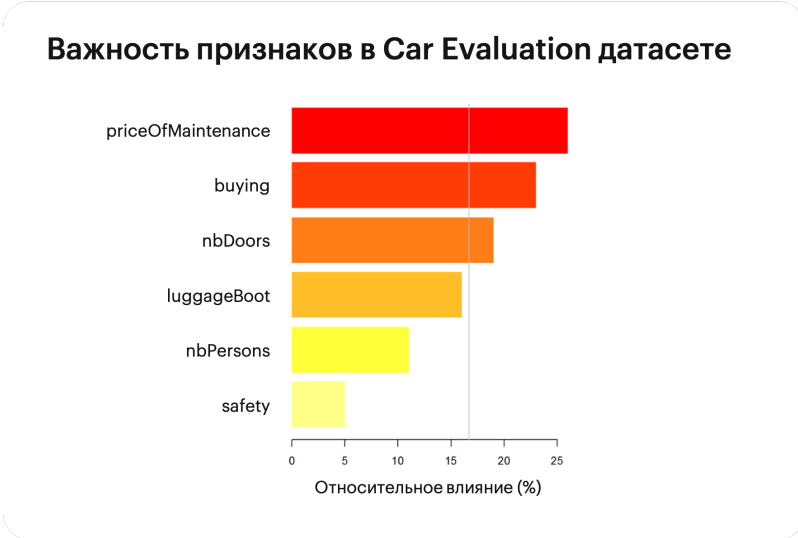


Рис. 18. Важности признаков по мере процентного уменьшения меры энтропии после деления. Источник: cran.r-project.org

ОПРЕДЕЛЕНИЕ

Вспомним формулу функционала качества, которая измеряет уменьшение в мере неопределённости в результате сплита. Чем сильнее признак уменьшает неоднородность региона, тем он считается важнее. Важность признака — сумма умень-

> шений неоднородностей с помощью этого признака при построении дерева.

Этот способ называется **MDI** (Mean Decrease in Impurity), или Gini importance, — именно его возвращает атрибут `feature_importances_`. Чтобы важности складывались в единицу, каждое уменьшение неопределённости обрабатывают:

1. дополнительно взвешивают на долю объектов, прошедших через узел;
2. нормируют.

Подводный камень MDI: смещение к признакам с большим числом значений. MDI считается по обучающей выборке. И поэтому завышает важность признаков, у которых много возможных порогов, — непрерывных и категориальных с большим числом уровней. Логика здесь простая — чем больше у признака точек разбиения, тем у него выше:

- шансов случайно уменьшить неопределённость;
- накапливается MDI, даже если на самом деле признак — это чистый шум.

Мы уже отмечали это среди недостатков деревьев. Как именно выглядит это смещение на эксперименте, показано на рисунке 19.

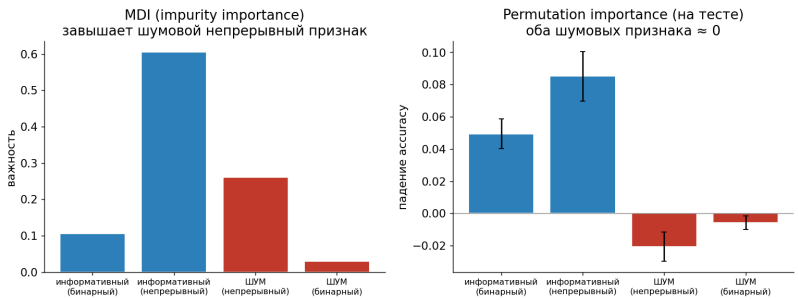


Рис. 19. Целевая переменная зависит только от двух информативных признаков. MDI (слева) завышает важность бесполезного непрерывного шума, тогда как permutation importance (справа) честно зануляет оба шумовых признака

На рисунке к двум информативным признакам добавлены два бесполезных шумовых: непрерывный (много уникальных значений) и бинарный. MDI присвоила непрерывному шуму важность около 0.26 — это больше, чем настоящему информативному бинарному признаку. Бинарный шум при этом почти равен нулю (0.03). Разница между двумя одинаково бесполезными шумами объясняется только числом порогов: у непрерывного их сотни, у бинарного — ровно один.

Посмотрев на дерево решений, мы уже сразу можем сделать предварительные выводы о важности признаков: более «высокие» сплиты обычно делят более важные признаки, так как именно первоначальные сплиты приводят к наибольшим падениям в неопределённости.

В Python использование атрибута `feature_importances_` из библиотеки scikit-learn поможет рассчитать важность признаков.

```
model = DecisionTreeClassifier()
model.fit(X, y)

# Получить важности признаков
importance = model.feature_importances_

```

```
# Можно расписать в виде текста и нарисовать гистограмму
for i, v in enumerate(importance):
    print(f'Feature: {i}, Score: {v:.5f}')
plt.bar([x for x in range(len(importance))], importance)
plt.show()
```

Permutation importance — это более честная альтернатива. Идея простая: модель обучают, замерив качество на отложенной выборке. А затем случайно перемешивают значения одного признака между объектами и замеряют, насколько просело качество:

- если признак важен — разрушение его связи с целевой переменной заметно ухудшит предсказания;
- если бесполезен — качество почти не изменится.

Такое падение качества и есть важность признака.

7.7. ТРИ ОТЛИЧИЯ PERMUTATION IMPORTANCE ОТ MDI:

- считается на отложенных, а не на обучающих данных — поэтому, не завышает важность признака просто за число порогов. На правой панели (рисунок 19), оба шумовых признака честно занулены. И даже чуть уходят в минус;
- модельно-независима — таким образом можно оценивать любую обученную модель, а не только дерево;
- имеет слабое место — коррелированные признаки: если два признака дублируют друг друга, перемешивание одного слабо вредит качеству (информация остаётся во втором) и важность обоих занижается.

```
from sklearn.inspection import permutation_importance

# model уже обучена; качество оцениваем на~отложенной выборке X_test, y_test
result = permutation_importance(model, X_test, y_test,
                                n_repeats=30,      # сколько раз перемешивать признак
                                    random_state=0)

for i in result.importances_mean.argsort()[::-1]:
    print(f'Feature {i}: {result.importances_mean[i]:.3f} '
          f'+/- {result.importances_std[i]:.3f}')
```

Что выбрать на практике:

- для интерпретации и отбора признаков стоит опираться на permutation importance (или продвинутые методы вроде SHAP);
- для быстрой, но смещённой первой оценки — MDI.

7.8. ГЛУБИНА ДЕРЕВА И ПЕРЕОБУЧЕНИЕ

В сквозном примере мы получили дерево со 100%-ной точностью на обучении и сразу заподозрили переобучение. Рассмотрим это дерево вживую: как меняется решающая граница на двумерных данных

при разной максимальной глубине (рисунок 20).

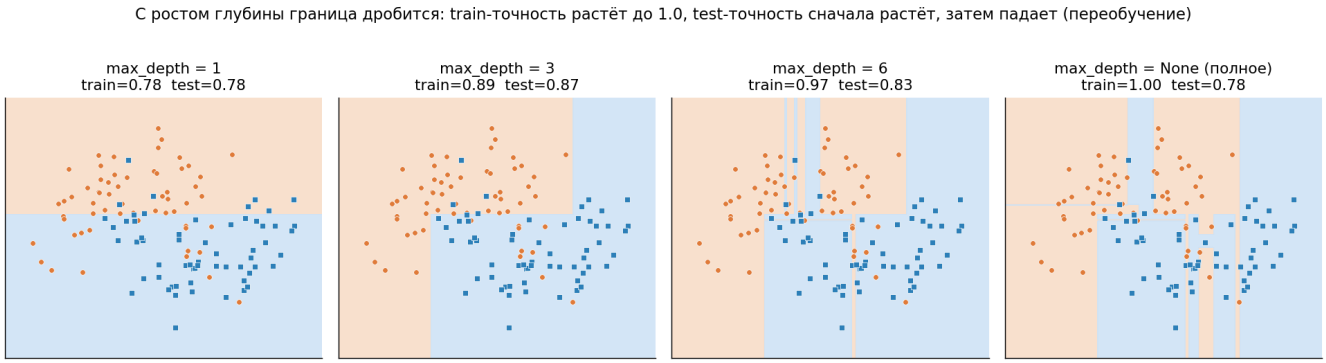


Рис. 20. Решающая граница дерева на «полумесяцах» при разной max_depth. Обучение — на маленькой шумной выборке, точность — на большой независимой. С ростом глубины train-точность растёт до 1.0, а test-точность сначала улучшается, потом падает

При `max_depth=1` дерево слишком простое — имеет одну горизонтальную границу, низкую точность обучающей и тестовой выборки. Это яркий пример недообучения.

При `max_depth=3` граница — это аккуратная «лесенка», а train и test высоки и близки. Это хороший баланс между простотой и качеством модели.

При глубине 6, а тем более у полного дерева — `max_depth=None` — граница дробится на узкие полоски, «гоняющиеся» за отдельными шумными точками. Train-точность доходит до 1.0, но test-точность падает — модель заучила шум обучающей выборки.

Это типичная картина для деревьев: train-ошибка монотонно убывает с глубиной, а test-ошибка сначала уменьшается, потом растёт. Поэтому глубину и другие ограничители (`min_samples_leaf`, `max_leaf_nodes`, `ccp_alpha`) подбирают по отложенной выборке или кросс-валидации, останавливаясь в «долине» тестовой ошибки.

7.9. НЕСТАБИЛЬНОСТЬ ДЕРЕВЬЕВ

Деревья решений — это крайне нестабильный алгоритм (non-robust). Малейшее изменение в обучающей выборке может привести к колоссальным изменениям в структуре обученного дерева.
Это говорит о высокой дисперсии (high variance) оценки зависимости между признаками и целевой переменной. Далее покажем, что высокая дисперсия оценки крайне нежелательна и может привести к увеличению ошибки предсказания тестовых данных (например, test MSE). Именно из-за высокой вариативности деревьев были разработаны **ансамблевые методы**: они сочетают неплохую предсказательную способность деревьев с возможностью снизить общую дисперсию модели.

8. Применение метода решающих деревьев

8.1. ПРЕИМУЩЕСТВА

- **Интерпретируемость, простота и white-box-особенность.** Дерево решений визуально легко представить и понять: каждая внутренняя вершина отражает условие на одном из признаков, а листья — итоговый прогноз. Деревья решений относятся к разряду white-box-моделей, у которых внутренняя структура и правила принятия решений понятны пользователю. Используя дерево, можно проследить путь от корня до листа («Если… то…»), что важно в задачах, где требуется интерпретируемость (медицина, финансы). В реальном мире во многих отраслях ценится интерпретируемость модели. Поэтому объяснимость/интерпретируемость — главное преимущество деревьев.
- **Непараметричность метода и отсутствие предположений о распределении данных.** Непараметричность деревьев позволяет моделировать зависимости любой сложности и избавляет от большей части предположений (assumptions) о распределении данных, виде зависимости.

  ВАЖНО

  Например, в отличие от линейных моделей, деревья не требуют нормальности признаков или линейной зависимости между ними. Более того, деревья могут работать с любыми типами данных (числовыми, категориальными) без сложной предобработки.

- **Гибкость и возможность преобразований.** Это фича алгоритма. Гибкость позволяет деревьям легко комбинироваться в ансамбли (Bagging, Random Forest, Gradient Boosting), повышая качество предсказаний без потери базового поведения.
- **Обработка пропусков.** Алгоритмы деревьев (особенно CART) известны способностью качественной обработки пропущенных значений без использования импутации (подстановки) данных. Многие деревья решений умеют расщеплять объекты с пропущенными значениями на несколько ветвей, а также вводят механизм суррогатных расщеплений для оценки оптимального расщепления признака с пропусками в данных.
- **Важность и автоматический отбор признаков.** При построении дерева наиболее информативные признаки оказываются ближе к корню, что показывает важность признаков и реализует implicit feature selection.
- **Невысокие вычислительные затраты на обучение и прогноз.** Благодаря жадному (поэтапному) алгоритму построения деревьев алгоритмическая сложность модели CART относительно невысока и может дополнительно снижаться и оптимизироваться. А прогноз для одного объекта сводится к простому последовательному обходу ветвей, что даёт быструю обработку даже в случае глубокого дерева.

8.2. НЕДОСТАТКИ

- **Склонность к переобучению (overfitting)** — одна из самых слабых сторон алгоритма и следствие его непараметрической природы. Известно, что глубокие деревья при отсутствии сдерживающих ограничений могут идеально подгоняться под шум тренировочных данных, теряя обобщающую способность на новых примерах. Поэтому важно тщательно настраивать гиперпараметры: максимальную глубину, минимальное число объектов в листе, критерии разбиения.
- **Невысокая точность и «кусочность» пространства решений.** Даже при эффективном контроле переобучения деревья уступают линейным моделям в задачах с линейными и нелинейными зависимостями.
- **Неустойчивость к небольшим изменениям в данных.** Из-за высокой дисперсии оценок модели даже небольшие изменения могут привести к построению совершенно другого дерева. Проблема усугубляется при переобучении на тренировочной выборке.
- **Смещение в сторону признаков с большим числом уровней.** При выборе сплита алгоритм предпочитает категориальный признак с большим количеством уникальных значений (больше возможных разбиений), даже если он менее информативен.
- **Требовательность к объёму данных при большой глубине** — это следствие непараметричности. Для стабильного обучения глубоких деревьев надо много примеров, в малых выборках модель легко подстраивается под шум.
- **Сложность оптимальной структуры.** Нахождение глобально оптимального дерева трудоёмко, поэтому используют жадные алгоритмы, которые могут пропускать лучшие и теоретически оптимальные решения.

9. Деревья vs линейные модели

Выбор оптимального алгоритма машинного обучения, который подстраивается под обучающую выборку и обладает обобщающей способностью, зависит от типа зависимости между признаками $X_1, \dots, X_p$ и целевой переменной Y.

Одна из главных задач — точно «угадать» вероятностное распределение нашей выборки. Зная это распределение, можно построить теоретически оптимальную модель с минимальными ошибками. Но почти никогда не известно, как выглядит такое распределение, и поэтому наша задача — оценить его с помощью выборки хотя бы примерно. Неверная оценка изначального распределения приводит к высоким ошибкам на тестовых данных. Такое бывает, когда, например, предполагаем линейную зависимость между переменными, чтобы построить линейную модель, хотя на деле эта зависимость явно нелинейна.

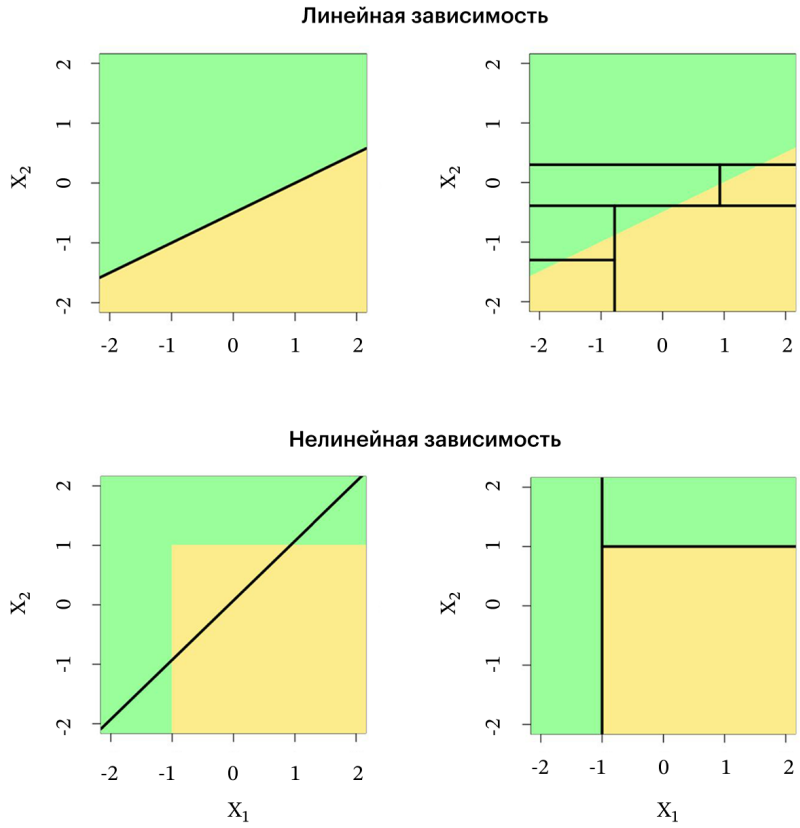


Рис. 21. Перформанс алгоритмов дерева и линейной модели (например, логистической регрессии) в двух случаях линейной и нелинейной зависимостей (задача классификации). Источник: ISL; Hastie, Tibshirani

На рисунке выше изображены два случая с линейной и нелинейной зависимостями в данных. Наглядно видно, к каким печальным результатам приводит выбор неверного алгоритма обучения.

Почему граница дерева — это всегда «лесенка». Дело в том, что осевые сплиты (вспомни рекурсивное бинарное разбиение) умеют проводить только горизонтальные и вертикальные разрезы. Поэтому любая граница дерева состоит из отрезков, параллельных осям, — даже в случае, когда «настоящая» граница гладкая или наклонная (рисунок 22).

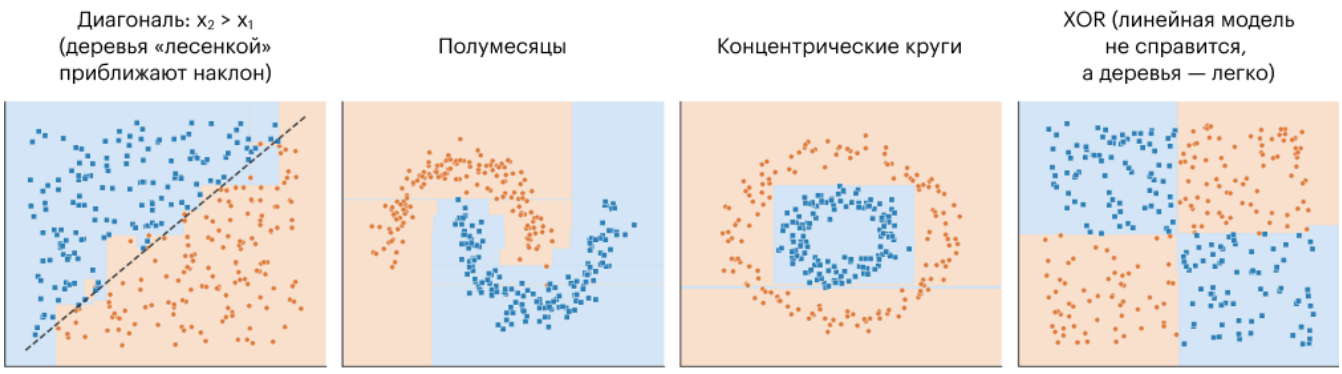


Рис. 22. Осевые сплиты дают только вертикальные и горизонтальные границы. Диагональные и гладкие границы дерево вынуждено приближать «лесенкой»; зато XOR делится двумя-тремя сплитами, где линейная модель бессильна

- **Диагональ.** Простую наклонную границу $x_2 > x_1$ линейная модель проводит одной прямой, а дереву приходится приближать её ступеньками — нужно много сплитов, и на краях всё равно остаются ошибки. Конкретная иллюстрация недостатка «кусочность пространства решений».
- **Полумесяцы и концентрические круги.** Гладкие криволинейные границы дерево обводит прямоугольными «коробками» — это узнаваемая ступенчатая аппроксимация.
- **XOR.** Здесь деревья отлично себя показывают. Достаточно двух-трёх осевых сплитов, чтобы идеально разбить плоскость на четыре квадранта, тогда как одна линейная граница принципиально не способна разделить XOR. Это случай сильной зависимости признаков — как раз там, где дерево бьёт линейную модель (см. таблицу ниже).

Итог. Если граница в данных близка к наклонной прямой — линейная модель опишет её проще и точнее. Если зависимость «прямоугольная» или с сильными взаимодействиями признаков — лучше справляется дерево решений.

Есть и другие критерии при выборе моделей.

Критерии	**Выбираем линейную модель**	**Выбираем дерево решений**
Зависимость	Если ближе к линейной	Если ближе к ступенчатой
Соотношение количества признаков к объектам	Если признаков $\gg$ объектов (так как регуляризаторы смягчают переобучение)	Если объектов $>$ признаков (так как дерево решений с огромным количеством признаков рассыплется)
Интерпретация результатов	Если нужна глобальная (какие признаки и как именно влияют на целевую переменную)	Если нужна локальная (например, почему этому клиенту отказываем в кредите)
Экстраполяция предсказаний	Если нужна	Если не нужна (так как дерево не умеет экстраполировать, за пределами обучающей выборки дерево предсказывает просто константу)
Взаимодействие между фичами	Если слабое	Если сильное
Специфичные данные	Если нет таких данных или их мало	Если есть большое количество категориальных фичей или пропущенных значений
Память и скорость алгоритма	Если критичны (например, при большом количестве данных или онлайн-обучении)	Если некритичны (непараметрические модели зачастую медленные)

ВАЖНО

Дерево можно представить в виде линейной модели.

Вспомним, что линейная регрессия — это модель вида

$$f(X) = \beta_0 + \sum_{j=1}^{p} \beta_j X_j.$$

В то же время дерево регрессии можно представить как

$$f(X) = \sum_{m=1}^{M} \beta_m \mathbb{I}(X \in R_m).$$

Таким образом, дерево представимо линейно: можно заменить пространство признаков $X_1, X_2, \ldots$ на признаки $\mathbb{I}(X \in R_m)$, то есть считать принадлежность объекта к каждому региону дерева за отдельный признак. Сформируется M dummy-признаков (принимающих 0–1 значения) для каждого из M регионов, и для каждого из этих признаков β_m будет значением целевой переменной на листе соответствующего региона.

Такой подход позволяет применять линейные модели в случае нелинейных данных. Понимание этого явления поможет, когда будем разбирать принципы работы бустинговых ансамблевых методов.

10. Если кратко

- → Дерево решений — алгоритм, который принимает решения, задавая последовательность вопросов по признакам; каждый внутренний узел — условие, каждый лист — итоговое предсказание.
- → Структура включает внутренние узлы (решающие условия), листья с константными ответами и рёбра, ведущие по исходам «Да/Нет».
- → Классический CART строится жадно через рекурсивное бинарное осевое разбиение: каждый сплит делит регион ровно на два подрегиона по одному признаку.
- → Высокая интерпретируемость: модель легко читается как цепочка вложенных if-else-правил и понятна доменным экспертам (кредитный скоринг, медицина, маркетинг и другое).
- → Обучение дерева — поиск признаков, порогов и значений в листьях через максимизацию функционала качества / минимизацию ошибки (меры неопределённости) на каждом шаге.
- → Деревья — непараметрические и сверхгибкие: число разбиений (параметров) не фиксировано заранее и может расти с данными, позволяя моделировать сложные нелинейности.
- → Деревья склонны к переобучению: без ограничений могут «поймать» шум, доведя почти каждый объект до отдельного листа, что ухудшает обобщение.
- → Выбор между деревом и линейной моделью зависит от характера зависимости (линейная vs нелинейная) и требований к интерпретации/точности.